THE UNIVERSITY OF DANANG
UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF MECHANICAL ENGINEERING

PBL 3: DESIGNING MICROCONTROLLER AND SENSOR APPLICATION DEVICE

Topic: Health Index Measuring Device

Teacher : **Assoc. Prof. Dr DANG PHUOC VINH**
Student : **NGUYEN HUU QUANG**
NGUYEN TRAN DUONG BAO
Class : **23CDT2 (23.05A)**

Da Nang, June 2026

PREFACE

In the context of the powerful Fourth Industrial Revolution (Industry 4.0), the remarkable advancements in microprocessor technology, embedded systems, and the Internet of Things (IoT) have become the core foundations for transformative changes in both daily life and industrial production. Intelligent control systems driven by microcontroller chips integrated with precision sensor networks are increasingly asserting their vital role. These systems optimize performance, enhance reliability, and unlock infinite potential for bridging the physical world with the digital environment.

The PBL2 course, "Design of Microcontroller and Sensor Applications," provides students with an initial, in-depth approach to the PIC16F877A microcontroller family and various practical sensors. Through this project, the team has not only refined their programming skills but also mastered the complete workflow from initial conceptualization and design to the fabrication of finished electronic circuit boards (PCBs).

To complete this project, the team applied specialized knowledge from core subjects such as "Microcontroller Engineering" and "Industrial Sensors." Furthermore, the project utilized professional software tools: CCS for embedded programming, Altium Designer for schematic and PCB layout design, and Visual Studio for developing the computer-based control and monitoring interface.

The team would like to express their deepest to Assoc. Prof. Dr Dang Phuoc Vinh. With his dedicated guidance, professional direction, and enthusiastic support throughout the implementation process, the team gained the motivation and technical insight necessary to resolve complex engineering challenges and successfully complete this project.

Da Nang, April 2026

Performing Students

Nguyen Huu Quang **Nguyen Tran Duong Bao**

TABLE OF CONTENTS

PREFACE **1**
TABLE OF CONTENTS **2**
CHAPTER 1: INTRODUCTION **4**
1.1. The Necessity of a Health Index Monitoring Device 4
1.2. What is the Health Index Measuring Device? 5
1.3. Some common health monitoring devices in practice. 6
CHAPTER 2: HARDWARE DESIGN **10**
2.1. Block Diagram 10
2.2. Hardware Overview 11
2.2.1. PIC 16F877A Microcontroller 11
a. Pinout PIC16F877A 11
b. General structure of PIC16F877A 14
c. Peripheral characteristics 15
2.2.2. Max232 16
2.2.3. USB to RS232 Converter 17
2.2.4. LCD 1602 Screen 17
2.2.5. Ultrasonic sensor (HC-SR04) 19
2.2.6. Loadcell & Module HX711 20
a. Loadcell 20
b. HX711 22
2.2.7. MAX30102 Sensor 23
2.2.8. Keypad 4x4 26
2.2.9. 220VDC – 12VDC Step-Down Converte 27
2.2.10. Webcam 27
2.3. Actual Wiring Diagram of the System 28
2.4. Altium Design 29
2.4.1. Introduction 29
2.4.2. Schematic 30
2.4.3. Layout Design (PCB) 31
CHAPTER 3: SOFTWARE DESIGN **32**
3.1. CCS C Compiler 32
3.1.1. Introduction to CCS C Compiler 32
3.1.2. Flowchart 32
a. Main flowchart 32
b. Weight Flowchart 36
c. Height Flowchart 37
d. BPM & SPO2 Flowchart 38

e. Keypad Flowchart 40
f. Start Measure Flowchart 41
g. INT TIMER2 42
3.1.3. Code PBL3_main.c 42
3.2. Visual Studio 51
3.2.1. Introdution to Visual Studio 51
3.2.2. User Interface 51
3.2.3. Code 52
CHAPTER 4: DISCUSSION AND CONCLUSION 55
4.1. System Prototype 55
4.2. User Guide 55
4.2.1. Step 1: System Initialization and Software Launch 56
4.2.2. Step2: QR Code Scanning and Data Entry 56
4.2.3. Step 3: Vital Signs Measurement and BMI Calculation 57
4.2.4. Step 4: History Logging, PDF Export, and Printing 58
4.3. Conclusion 59
4.3.1. Results 59
4.3.2. Techical Issues during Poduct Finalization 59
4.3.3. Future Development 60
REFERENCES 61

CHAPTER 1: INTRODUCTION

1.1. The Necessity of a Health Index Monitoring Device

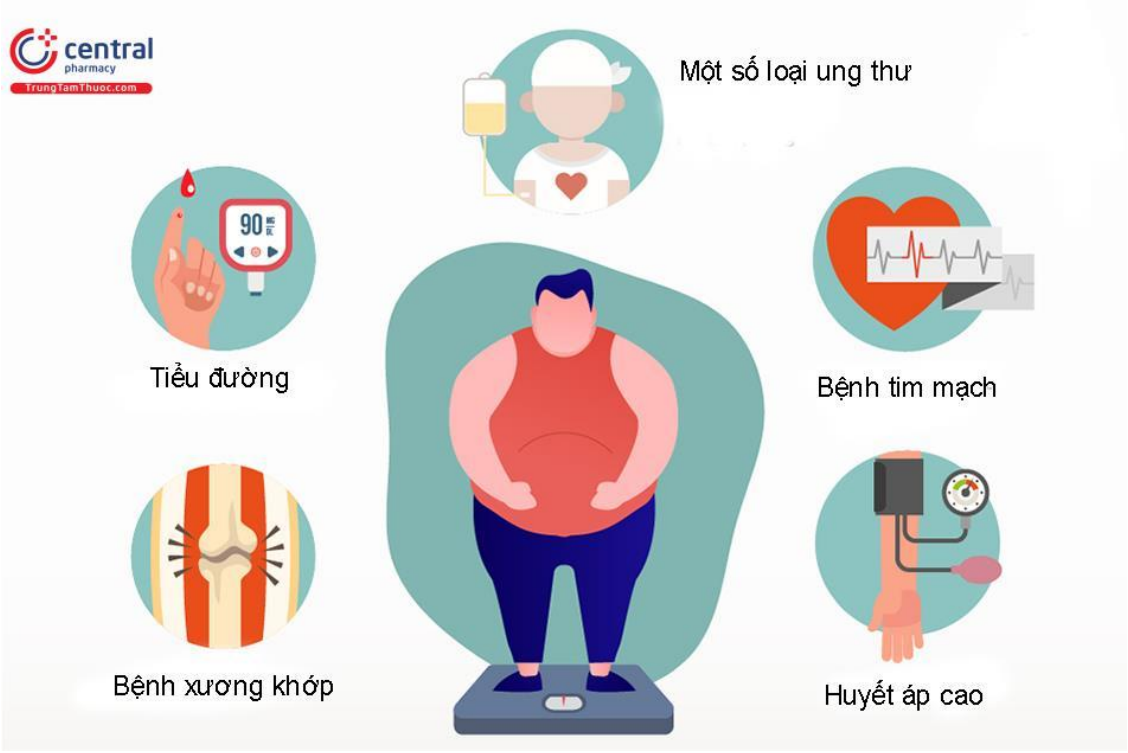


Figure 1. Potential health risks of obesity (Source: Central Pharmacy)

Nowadays, most people don't really pay attention to their health metrics. We usually only start worrying about our weight when we notice our bodies are out of shape. However, keeping track of these numbers is very important it helps us understand our bodies better and catch any sudden changes early so we can see a doctor in time.

At small hospitals, the step of measuring a patient's body metrics is often skipped entirely. On the other hand, at larger hospitals, while nurses do take these measurements before a check-up, the process takes quite a long time because everything is still done manually on paper.

That’s why this device was created. It measures blood oxygen levels (SpO2), heart rate, and BMI, then automatically compares the results with WHO standards to provide an immediate health assessment. Instead of writing everything down by hand, the system processes and saves all data directly to a computer via an ID card scanner. This automation significantly cuts down on administrative time, allowing patient information and measurement results to be updated to the hospital’s database instantly and accurately.

1.2. What is the Health Index Measuring Device?



Figure 1.2. Using Scale and Stadiometer for BMI Measurement (Source: Maxcare Home)

The health monitoring device is an integrated system that measures height and weight to assess nutritional status (BMI), alongside blood oxygen levels (SpO2) and heart rate.

The device includes several features to support both nurses and users, such as:

- ID Card QR Scanning: Instantly retrieves and saves patient information.
- Computer-Based Control: Allows the device to be operated directly through a computer.
- Reduced Staffing Needs: Minimizes the number of nurses required to guide patients through the measurement process.
- WHO Standards: Utilizes World Health Organization guidelines to accurately diagnose the body's physical condition.

1.3. Some common health monitoring devices in practice.



Figure 1.3. Tanita BH-200 Semi-Automatic BMI Scale and Stadiometer (Source: Maxcare Home)

The Tanita BH-200 is a high-precision digital scale designed for measuring height, weight, and Body Mass Index (BMI) in school health screenings, hospitals, and medical facilities.

Advanced Height Measurement: Utilizes Camera Dot Code technology to provide highly accurate results. The integrated scale allows for visual verification of height measurements.

Tare Function: Features a built-in "Clothing Weight Offset" function to subtract known clothing weight for a more precise net body mass reading.

Data Integration: Supports data transmission to a PC via serial output through USB or RS232C ports.



Figure 1.4. SECA Electronic Column Scale (Source: Drvip.vn)

SECA 797 – EMR validated column scale with eye – level display and Wifi function. Medical – grade, robust scale for years of use with high patient volume. Digital measuring rod with an extra long measuring range. Two – line display shows height and weight. EMR validated with built – in WLAN module. Damping system and Auto_HOLD guarantee quick and precise weighing.
With its WiFi interface the seca 797 can be integrated directly into your EMR system.



Figure 1.5. InBody 270(Source: Inbody.com)

Additionally, more advanced and modern body composition analyzers, such as the InBody 270 and 770, provide comprehensive result sheets including: Muscle Mass, Body Fat Mass, Weight, Protein, Minerals, Total Body Water (TBW), BMI, Percent Body Fat (PBF), and Waist-Hip Ratio (WHR).

After completing the measurements, our Health Index Measuring Device provides specific indicators, benchmarks them against WHO standards, and delivers the following diagnostic evaluations:

- BMI Analysis: Based on the measured weight and height, the system automatically calculates the Body Mass Index (BMI). The diagnostic classification is then determined according to WHO standards [Figure 1.6].



Figure 1.6. The Body Mass Index - BMI (Source: Bass Medical Group)

- Blood Oxygen Saturation (SpO2): The system measures the percentage of oxygen-carrying hemoglobin in the blood. The results are displayed and evaluated based on clinical safety thresholds [Figure 1.7].
- Heart Rate (Pulse): The device detects the number of heartbeats per minute (BPM). This data is then analyzed to determine if the user's resting heart rate falls within a healthy range [Figure 1.8].

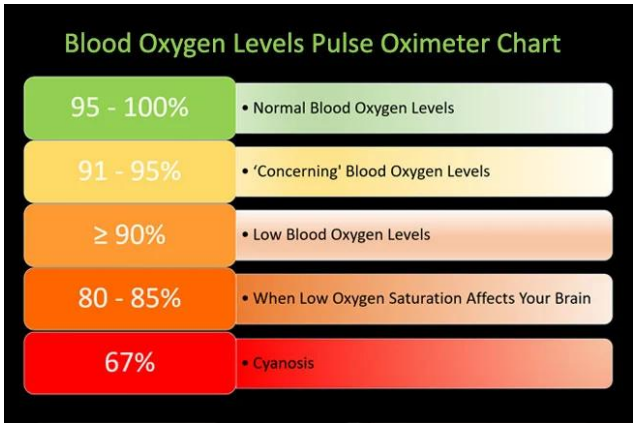


Figure 1.7. SpO2 Index (Source: FaCare)

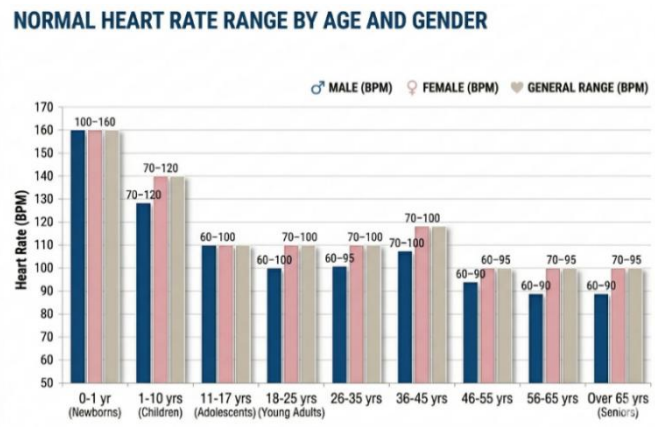


Figure 1.8. Heart Rate Index (Source: Internet)

CHAPTER 2: HARDWARE DESIGN

2.1. Block Diagram

The Health Index Measuring Device is an integrated system designed to measure weight and height, alongside heart rate and blood oxygen saturation (SpO2). The device captures input data using an HC-SR04 ultrasonic sensor for height, a Load Cell for weight, and a MAX30102 sensor for SpO2 and heart rate monitoring. All data is processed by a PIC16F877A microcontroller, with outputs displayed on an LCD 1602 and signal LEDs. Furthermore, the device communicates with a PC via the UART – RS232 protocol. A standout feature is the integration of a Webcam to scan QR codes from National ID cards, enabling automated patient information entry and systematic medical record storage.

The following figure illustrates the block diagram of the system:

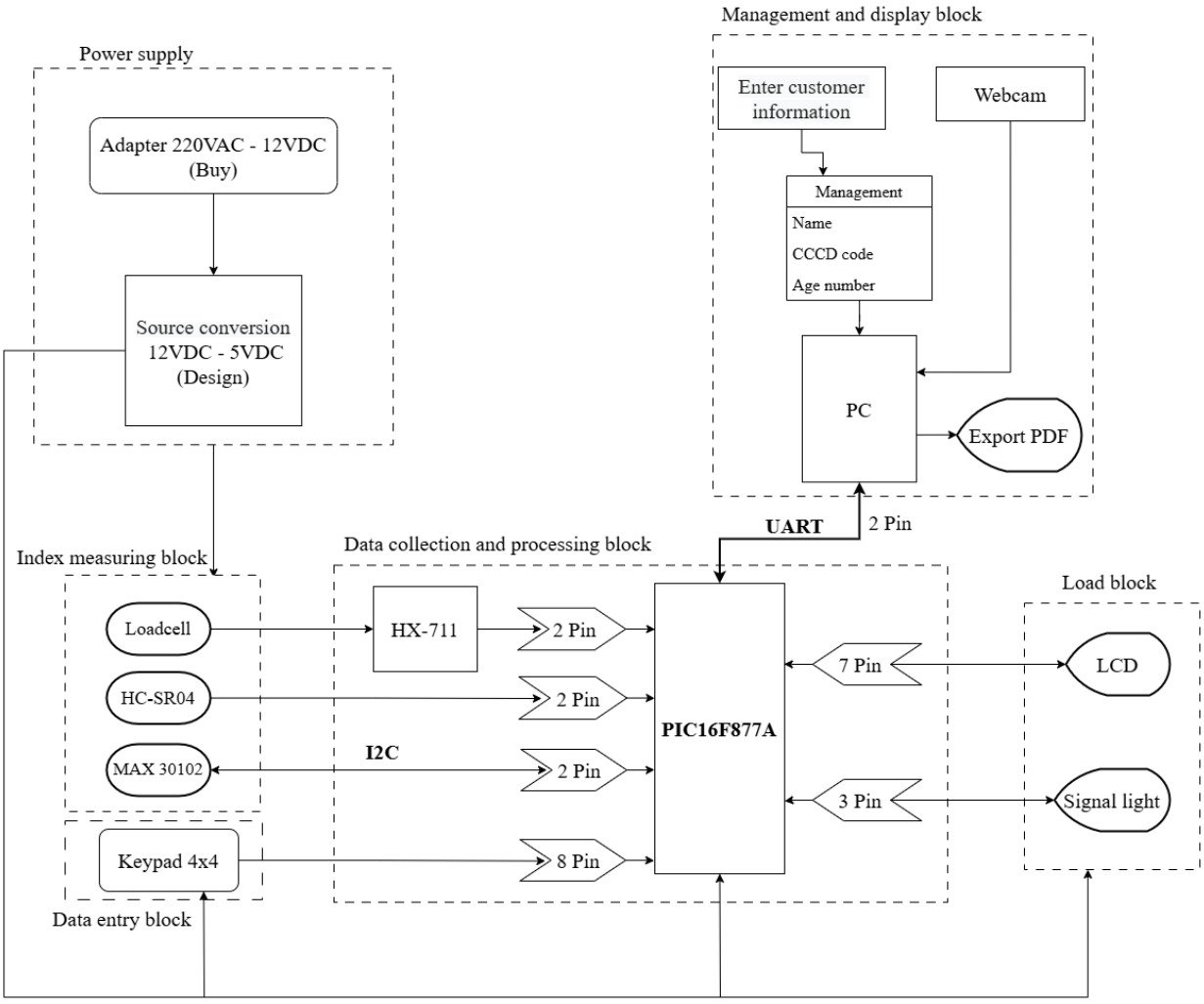


Figure 2.1. Block Diagram System

2.2. Hardware Overview

2.2.1. PIC 16F877A Microcontroller



Figure 2.2. PIC 16F877A (Source: Internet)

The PIC16F877A is a microcontroller from the PIC16F family, manufactured by Microchip. This chip is widely used in both academic environments and industrial automation devices.

PIC16F877A is a 40 – pin PIC Microcontroller and is used mostly in embedded projects and applications. It has five ports starting from port A to port E. It has three timers of which two are 8-bit timers and one is 16-bit timers. It supports many communication protocols such as serial protocol, parallel protocol. The PIC16F877A supports both hardware pin interrups and timer interrupts.

a. Pinout PIC16F877A

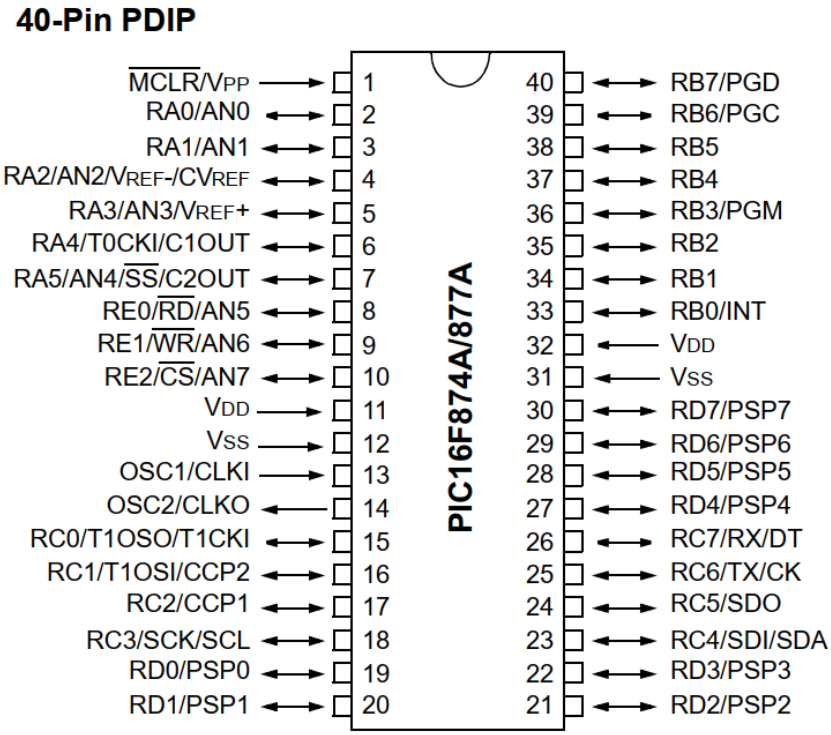


Figure 2.3. Pinout PIC16F877A [3]

Table 1.1. Funtion of Pinouts of PIC16F877A [2][3]

Pin	Name	Description
1	MCLR/VPP	– MCLR: Low level reset operation (Master clear pin external reset) – VPP: programmable pressure input
2	RA0/AN0	– RA0: output / import number – AN0: analog inpu
3	RA1/AN1	– RA1: number export/import – AN1 : analog input
4	RA2/AN2/VREF-/CVREF	– RA2: number export/import – AN2: analog input – VREF - : standard voltage input (low) of the A/D unit
5	RA3/AN3/VREF+	– RA3: number export/import – AN3 : analog input – VREF+: standard (high) voltage input of the A/D unit
6	RA4/TOCKI/C1OUT	– RA4: Export/import numbers – TOCKI: external clock pulse input for timer0 – C1 OUT: Comparator 1 output
7	RA5/AN4//SS /C2OUT	– RA5: number export/import – AN4 : analog input 4 – SS: secondary SPI selection input – C2 OUT: comparator output 2
8	RE0//RD/AN5	– RE0: import and export numbers – RD: controls the reading at the parallel branch port – AN5: analog input
9	RE1//WR/AN6	– RE1: number export/import – WR: controls the recording in the parallel branch port – AN6 : analog input
10	RE2//CS/AN7	– RE2: number export/import – CS: The chip selects control at the parallel branch port – AN7: analog
11	VDD	Power pin of PIC
12	VSS	Ground pin
13	OSC1/CLKI	– Input to quartz oscillations or external clock pulses. – OSC1: input to quartz oscillations or external clock pulses. Schmit trigger input when configured in RC mode; another way of CMOS. – CLKI: external pulse source input. Always combined with OSC1 function.
14	OSC2/CLKO	Quartz oscillating input or clock pulse – OSC2: Quartz oscillation output. Connect to quartz or resonator.

		– CLKO : in RC mode, the output of OSC2, equal to the frequency of OSC1 and indicates the speed of the instruction cycle.
15	RC0/T1 OCO/T1CKI	– RC0: digital export/import – T1OCO: Timer oscillator input 1 – T1CKI: external clock pulse input Timer 1
16	RC1/T1OSI/CCP2	– RC1: digital export/import – T1OSI: timer oscillator input 1 - CCP2: Capture 2 input, compare 2 output, PWM2 output
17	RC2/CCP1	– RC2: digital export/import – CCP1: Capture input 1, compare output 1, PWM1 output
18	RC3/SCK/SCL	– RC3: digital export/import – SCK: synchronous serial clock clock input / output of SPI mode – SCL: synchronous serial clock pulse input / output of I2C mode
19	RD0/PSP0	– RD0: digital export/import – PSP0: parallel branch port data
20	RD1/PSP1	– RD1: digital export/import – PSP1: parallel branch port data
21	RD2/PSP2	– RD2: digital export/import – PSP2: parallel branch port data
22	RD3/PSP3	– RD3: digital export/import – PSP3: parallel branch port data
23	RC4/SDI/SDA	– RC4: digital export/import – SDI: data into SPI – SDA: export/import data into I2C
24	RC5/SDO	– RC5: digital export/import – SDO: data out SPI
25	RC6/TX/CK	– RC6: digital export/import – TX: USART asynchronous transmission – CK : USART synchronous pulse
26	RC7/RX/DT	– RC7: digital export/import – RX: received USART disagreement – DT: USART synchronous data
27	RD4/PSP4	– RD4: digital export/import – PSP4: parallel branch port data
28	RD5/PSP5	– RD5: digital export/import – PSP5: parallel branch port data
29	RD6/PSP6	– RD6: digital export/import – PSP6: parallel branch port data
30	RD7/PSP7	– RD7: digital export/import – PSP7: parallel branch port data
31	VSS	Ground pin
32	VDD	Power pin of PIC

33	RB0/INT	– RB0: number export/import – INT: external interrupt
34	RB1	Export/import numbers
35	RB2	Export/import numbers
36	RB3	– RB3: number export/import – ICPS low voltage programmable pin
37	RB4	– Export/import numbers – PortB interrupt
38	RB5	– Export/import numbers – PortB interrupt
39	RB6/PGC	– RB6: number export/import – PGC: ICSP programmable differential circuit and clock pulse – PortB interrupt
40	RB7/PGD	– RB7: number export/import – PGD: differential circuits and ICSP programmable data

b. General structure of PIC16F877A

The general architecture of the PIC16F877A microcontroller is characterized by the following features [1]:

- 8-bit CPU based on the Modified Harvard Architecture.
- Flash and ROM memory options ranging from 256 bytes to 256 Kbytes.
- I/O ports with logic levels typically from 0V to 5.5V, corresponding to Logic 0 and Logic 1.
- Standard serial peripheral interfaces, including synchronous/asynchronous USART, AUSART, and EUSART.
- Support for various oscillation frequencies (refer to Figure 2.4).
- Integrated ADC (Analog-to-Digital Converter) with 10/12-bit resolution.
- Built-in Voltage Comparators.
- Two CCP modules (Capture/Compare/PWM).
- Part of the PIC16Fxxx family with an instruction set consisting of 35 instructions, each 14 bits in length.
- All instructions execute in one machine cycle, except for program branches (subroutines), which require two machine cycles.
- One instruction cycle consists of 4 clock pulses. For example, if a 4MHz crystal oscillator is used, the instruction frequency will be 1MHz, corresponding to a cycle time of 1us.
- Flash Program Memory: 8K x 14-bit capacity, with an endurance of up to 100,000 write/erase cycles.
- Data RAM: 368 bytes of capacity.
- EEPROM: 256 bytes of capacity, with an endurance of up to 1,000,000 write/erase cycles and data retention exceeding 40 years.
- Operating Voltage: 4.0V to 5.5V DC (refer to Figure 2.4).

- Power-saving Sleep mode for low power consumption.
- Five I/O ports (labeled A, B, C, D, and E) providing a total of 33 pins.
- Support for communication protocols including USB, Ethernet, CAN, LIN, and IrDA.

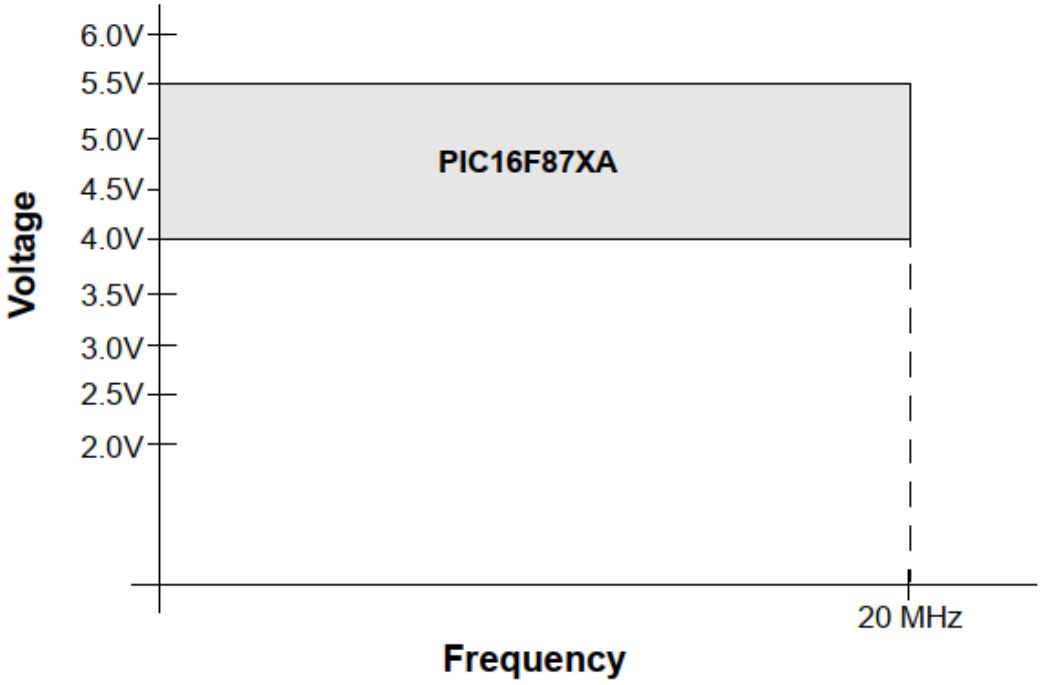


Figure 2.4. Voltage suplie for PIC16F877A [3]

c. Peripheral characteristics

Includes the following functional blocks [1]:

- Timer0: 8-bit counter with 8-bit frequency splitter.
- Timer1: 16-bit counter with frequency splitter, can perform counting function based on peripheral clock pulse as soon as the microcontroller is operating in sleep mode.
- Timer2: 8-bit counter with frequency splitter, postcaler. who sets Capture/compare/modulate pulse mode.
- SSP (Synchronous Serial Port), SPI and I2C serial interface standards. USART serial communication standard.

2.2.2. Max232

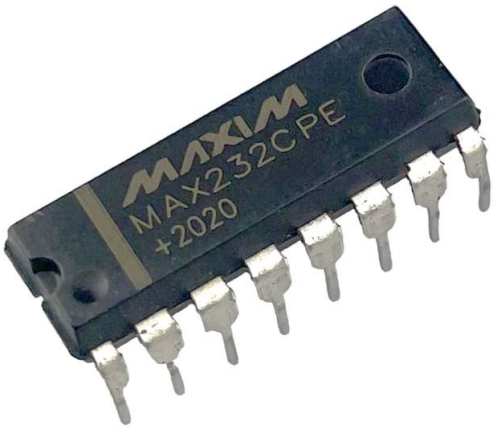


Figure 2.5. IC Max232 (Source: Internet)

The MAX232 device is a dual driver/receiver that includes a capacitive voltage generator to supply TIA/ EIA-232-F voltage levels from a single 5V supply. Each receiver converts TIA/EIA-232-F inputs to 5V TTL/CMOS levels. These receivers have a typical threshold of 1.3 V, a typical hysteresis of 0.5V, and can accept ±30V inputs. Each driver converts TTL/ CMOS input levels into TIA/EIA-232-F levels. Pin Configuration and Functions:

PIN		TYPE	DESCRIPTION
NAME	NO.		
C1+	1	—	Positive lead of C1 capacitor
VS+	2	O	Positive charge pump output for storage capacitor only
C1-	3	—	Negative lead of C1 capacitor
C2+	4	—	Positive lead of C2 capacitor
C2-	5	—	Negative lead of C2 capacitor
VS-	6	O	Negative charge pump output for storage capacitor only
T2OUT	7	O	RS232 line data output (to remote RS232 system)
R2IN	8	I	RS232 line data input (from remote RS232 system)
R2OUT	9	O	Logic data output (to UART)
T2IN	10	I	Logic data input (from UART)
T1IN	11	I	Logic data input (from UART)
R1IN	13	I	RS232 line data input (from remote RS232 system)
T1OUT	14	O	RS232 line data output (to remote RS232 system)
GND	15	—	Ground
V_{CC}	16	—	Supply Voltage, Connect to external 5V power supply

Figure 2.6. Pinout IC MAX232 [12]

2.2.3. USB to RS232 Converter



Figure 2.7. USB to RS232 Converter (Source: Internet)

A connection port on the computer acts as an interface or simply a communication point between the computer and other peripheral devices . Some of the most common peripherals are mice, keyboards, monitors or other display devices, printers, speakers, flash drives, etc.

The main use of a computer port is to act as a mount point, where cables from peripheral devices can be plugged in , through which , allowing data to travel to and from the device.

In computers, communication ports can be divided into two types based on the type or protocol used to connect . They are Serial Ports , Parallel Ports.

A serial port is a physical layout and interface through which peripheral devices can be connected using a serial protocol. This process involves transmitting digital communication one bit at a time over a single transmission line. The most common type of serial port is the D-Subminiature (or D-sub) connector, which is typically used to carry RS-232 signals.

2.2.4. LCD 1602 Screen

LCD is used to display status or parameters. LCD has 16 pins in which 8 data pins (D0 – D7) and 3 control pins (RS, RW, EN). The remaining 5 pins are used to provide power and backlight for 16x02 LCD. The control pin make it easy to configure the LCD in command mode or data mode.



Figure 2.8. LCD screen (Source: hshop.vn)

The LCD 1602 is a green-backlight display powered by the HD44780 controller. It can display 2 lines of 16 characters each. This module is highly durable, widely popular, and easy to use.

Technical Specifications:

- Operating Voltage: 5V DC.
- Dimensions: 80 x 36 x 12.5 mm.
- Display Style: Black text on a green background.
- Pin Pitch: 2.54 mm (0.1 inch), making it breadboard-friendly.
- Ease of Connection: Pin names are labeled on the back for convenient wiring.
- Backlight: Integrated LED backlight, with adjustable brightness via a potentiometer or PWM to save power.
- Control Interface: Can be operated using only 6 signal wires (4-bit mode).
- Character Set: Built-in support for English and Japanese characters.

Table 1.2. Funtion of Pinouts of LCD 1602

Pin	Symbol	Description	Value / Function
1	VSS	GND	0VDC
2	VCC		5VDC
3	V0	Contrast Adjustment	
4	RS	Register Select	RS = 0 (Low): Select Instruction Register RS = 1 (High): Select Data Register
5	R/W	Read/Write Select	R/W = 0: Write mode R/W = 1: Read mode
6	E	Enable	
7	D0	Data Bus Line	8 bit: D0-D7
8	D1		
9	D2		
10	D3		
11	D4		
12	D5		
13	D6		
14	D7		
15	A	Backlight Anode	0 – 5 VDC
16	K	Backlight Cathode	0VDC

2.2.5. Ultrasonic sensor (HC-SR04)



Figure 2.9. HC-SR04 Ultrasonic sensor (Source: Internet)

Ultrasonic sensors work by emitting ultrasonic waves with a high frequency that is too high for a human to hear. These waves travel through the air with the speed of sound, roughly 343m/s. . If there is an object in front of the sensor, the sound waves get reflected back and the receiver of the ultrasonic sensor detects them. By measuring how much time passed between sending and receiving the sound waves, the distance between the sensor and the object can be calculated.

The HC-SR04 is an inexpensive, easy to use ultrasonic distance sensor, with a range of 2cm to 400 cm. It is commonly used in obstacle avoiding robots and automation projects.

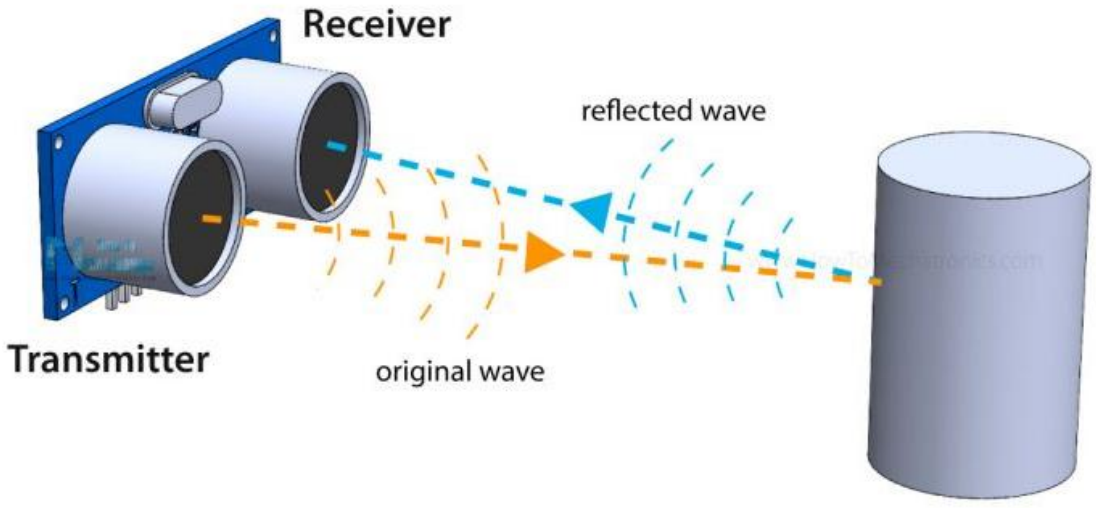


Figure 2.10. HC-SR04 Ultrasonic sensor operating principle [6]

Distance Caculating:

$$\text{Distance} = \frac{T \times v}{2}$$

Where:

- Distance**:** distance between the sensor and the object is Actually – (cm)
- T**:** is the the time it takes for the ultrasonic wave to travel to the object and back (measured in seconds or microseconds – µs).
- v**:** is the speed of sound in air – cm/µs (approximately 343 m/s at 25°C)
- Division by 2 is required because the measured time includes both the outgoing and returning path of the sound wave.

Table 1.3. HC-SR04 electrical characteristics [5][6]

Description	**Value**
Power Supply	5VDC
Working Current	15mA
Working Frequency	40kHz
Maximum Range	400cm
Minimum Range	2cm
Measuring Angle	30°
Effectual Angle	<15°
Resolution	0.3 cm
Trigger Input Signal	10uS TTL pulse
Echo Output Signal	TTL pulse proportional to the distance range
Dimensions	45 20 x 15 mm

2.2.6. Loadcell & Module HX711

a. Loadcell



Figure 2.11. Beam Loadcell (Source: Internet)

When a Loadcell is under a mechanical load, it undergoes deflection (bending). At the point where the Strain Gauge is attached, the material experiences either tension (stretching) or compression.

Strain Gauge Sensor:

A Strain Gauge consists of thin metal foils or conductors arranged in a pattern so that their electrical resistance changes when they are compressed or stretched. When mounted onto an object (typically a metal beam), its resistance varies based on the stress applied to that object. It converts mechanical force into a measurable change in electrical resistance.

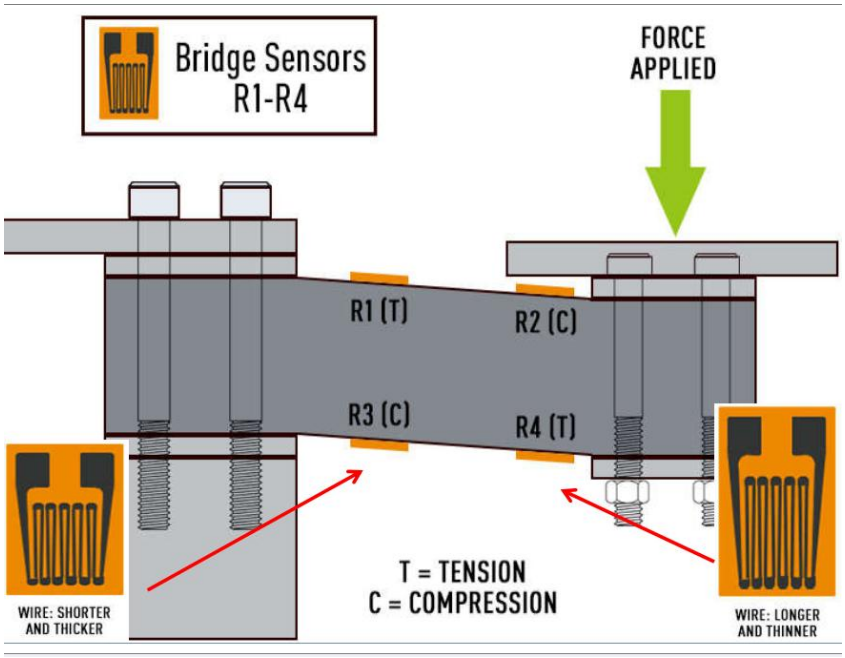


Figure 2.12. Strain gauges mounted on the load cell surface [4]

Based on the sensitivity specifications provided by the manufacturer, the Maximum Output Voltage is calculated using the following formula:

$$V_{out_{max}} = C \times V_{in}$$

Where:

- $V_{out_{max}}$: Maximum output voltage (mV).
- C : Loadcell sensitivity (mV/V).
- V_{in}: Input excitation voltage (V).

To determine the specific output voltage relative to the actual weight placed on the scale, use the following formula:

$$V_{out} = \frac{W_{actual}}{W_{max}} \times C \times V_{in}$$

Where:

- V_{out} : Output voltage (mV).
- C : Loadcell sensitivity (mV/V).
- V_{in}: Input excitation voltage (V).
- W_{actual}: Actual mass/weight placed on the scale (kg).
- W_{max}: Maximum rated capacity of the loadcell (kg).

Table 1.4. Load Cell Technical Specifications

Description	Wire color	Value
Excitation + (E+)	Red	5VDC
Excitation - (E-)	Black	0VDC
Output + (O+)	White	xxx mV
Output – (O-)	Green	xxx mV
YLW	Yellow	GND
Type	-	1kg
Safe overload	-	150%

b. HX711

The voltage level output by the load cell is very small, only a few mV (for example: 20 mV). This is too small for standard microcontrollers (PIC) to read directly and accurately. Therefore, we must use the HX711 to both amplify the signal thousands of times and convert the signal from Analog to Digital.

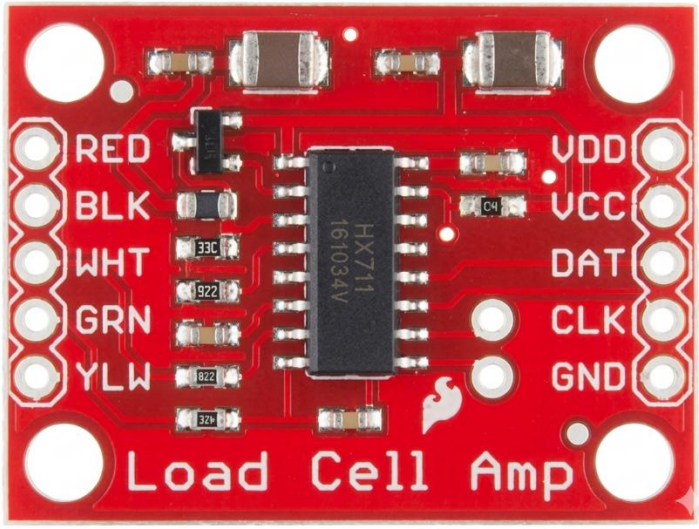


Figure 2.13. Module HX711 [7]

HX711 is a high-precision 24-bit analog-to-digital converter (ADC) and amplifier, designed for use in electronic scales and industrial electronic applications.

Technical Specifications:

- Operating Voltage: 2.7 – 5V DC
- Current Consumption: < 1.5mA
- Sampling Rate: 10 – 80 SPS (selectable)
- Resolution: 24-bit ADC
- Voltage Resolution: 40mV
- Dimensions: 38 x 21 x 10 mm

Table 1.5. Pinout HX711 [7][8]

Pinout HX711	Function
RED (Input E+)	5VDC Power Supply for Load Cell
BLK (Input E-)	0VDC (Ground) for Load Cell
WHT (Input O+)	Positive Differential Signal from Load Cell
GRN (Input O-)	Negative Differential Signal from Load Cell
YLW (GND)	GND
VDD	Input 3.3VDC
VCC	Input 5VDC
DAT (Dout)	Serial Data Output
CLK (SCK/Clock)	Serial Clock Input (Power Down and Serial Clock)

2.2.7. MAX30102 Sensor

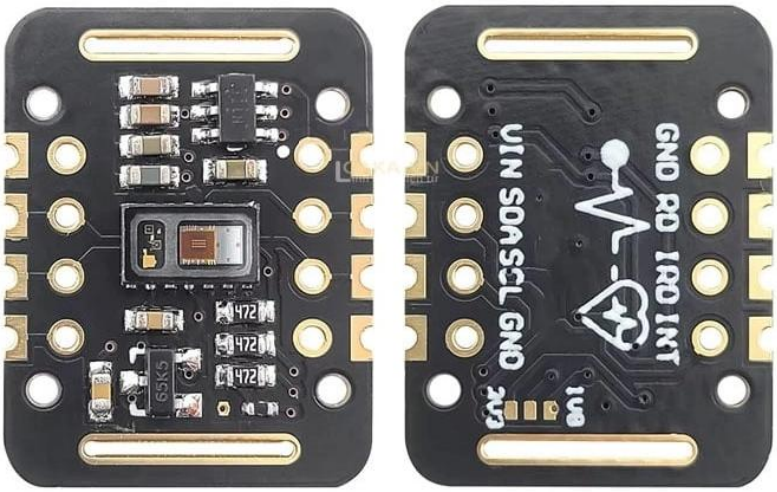


Figure 2.14. Module Max30102 (Source: Internet)

The MAX30102 sensor integrates a Red LED, an Infrared (IR) LED, and a photodetector. The Red LED operates at a wavelength of 660 nm, while the IR LED operates at 880 nm. The sensor functions by emitting light through the skin and measuring the light reflectance using its internal photodetector. The operation of the MAX30102 can be categorized into two primary functions: Heart Rate Monitoring and Blood Oxygen Saturation (SpO2) Measurement.

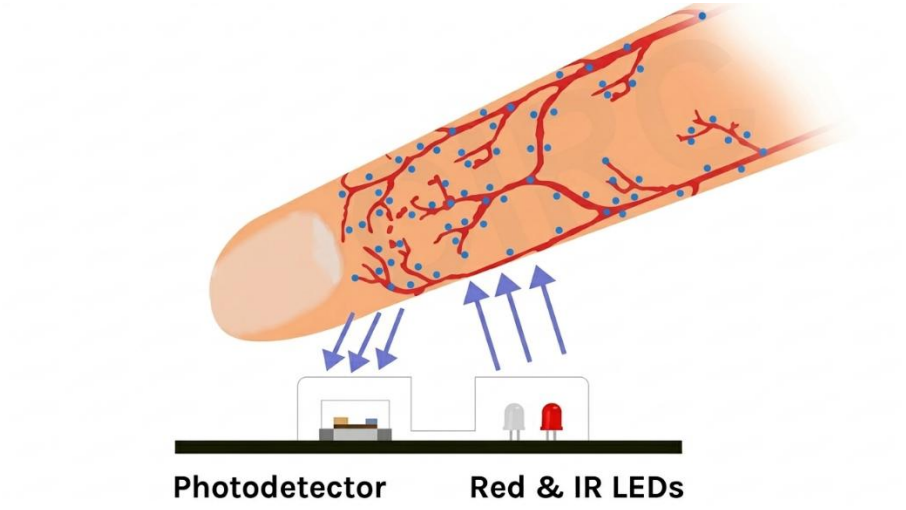


Figure 2.15. Operating Principle of the MAX30102 [11]

This method relies on the infrared (IR) absorption properties of hemoglobin in the blood. Higher hemoglobin concentrations correlate with increased IR light absorption. As the heart pumps blood through the capillaries in the fingertips, the blood volume changes with each beat, causing corresponding shifts in light absorption and reflection. These variations produce an oscillatory waveform, which is then analyzed to calculate an accurate heart rate.

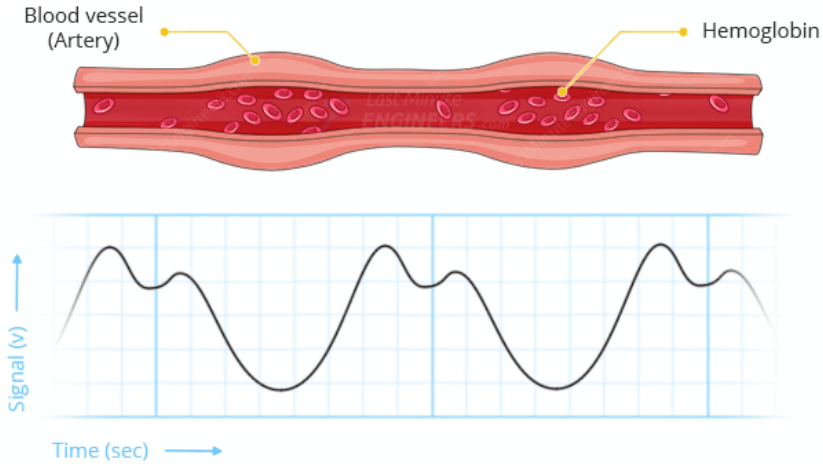


Figure 2.16. Heart Rate Measurement Principle [10]

- Blood Oxygen Saturation (SpO2) Measurement: This method is based on the differential light absorption properties of oxygenated hemoglobin (HbO2) and deoxygenated hemoglobin (Hb).
  - Red Light: Strongly absorbed by deoxygenated hemoglobin (Hb).
  - Infrared (IR) Light: Strongly absorbed by oxygenated hemoglobin (HbO2).

The device compares the absorption ratios of these two wavelengths to accurately calculate the blood oxygen saturation level.

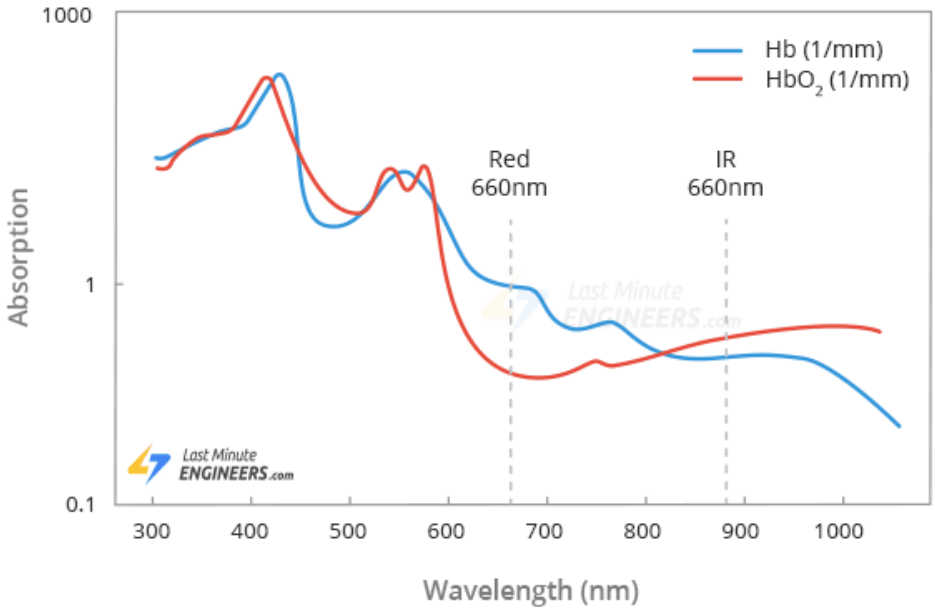


Figure 2.17. SpO2 Measurement Principle [10]

Technical Specifications [10]

- Power supply: 3.3 to 5.5V
- Current draw: ~ 600µA (during measurements)
  ~0.7µA (during standby mode)
- Red LED Wavelength: 660nm
- IR LED Wavelength: 880nm
- Temperature Range: -40˚C to +85˚C
- SpO2 Measurement Range: 70% - 100%
- Dimensions: 20.6 x 15.5 x 3 (mm)
- Communication Protocol: I2C (Inter-Integrated Circuit)

Table 1.6. pinout module Max30102 [10]

Pin Name	Type	Description
VIN	Power	Power supply input (3.3V - 5V)
GND	Ground	Reference ground
SCL	Input	I2C Serial Clock line
SDA	I/O	I2C Serial Data line
INT	Output	Active-low interrupt signal
IRD	Input	Manual control for the IR LED
RD	Input	Manual control for the Red LED

Functional Diagram:

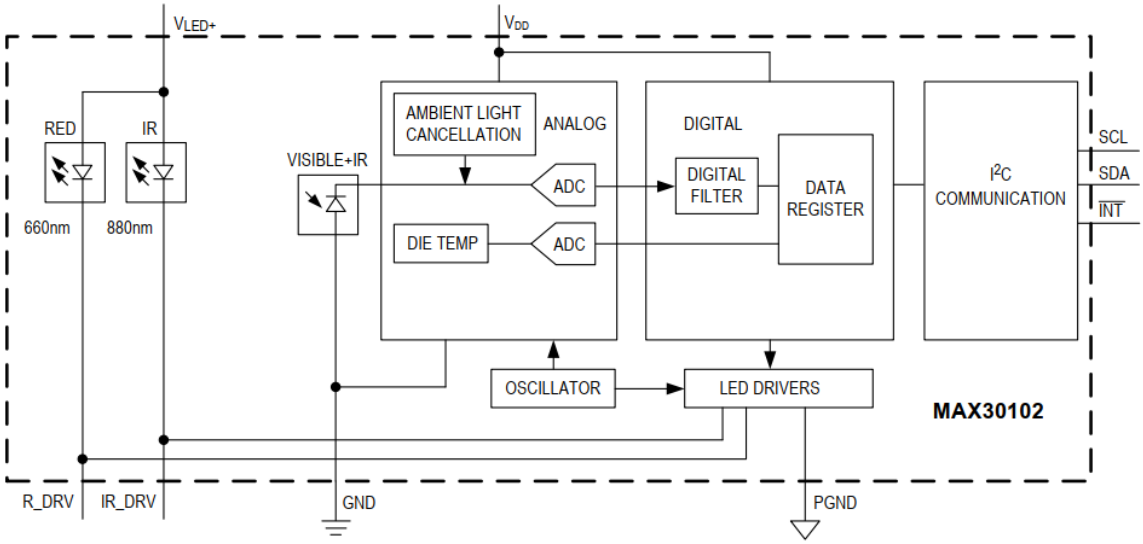


Figure 2.18. Functional Diagram [9]

2.2.8. Keypad 4x4

Matrix keypads use a combination of four rows and four columns to provide button states to the host device, typically a microcontroller. Underneath each key is a pushbutton, with one end connected to one row, and the other end connected to one column. These connections are shown in Figure 2.18

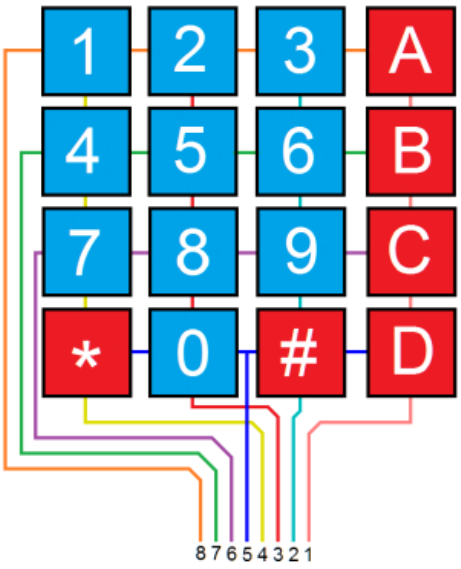


Figure 2.19. Matrix Keypad Connections [13]

In order for the microcontroller to determine which button is pressed, it first needs to pull each of the four columns (pins 1-4) either low or high one at a time, and then poll the states of the four rows (pins 5-8). Depending on the states of the columns, the microcontroller can tell which button is pressed.

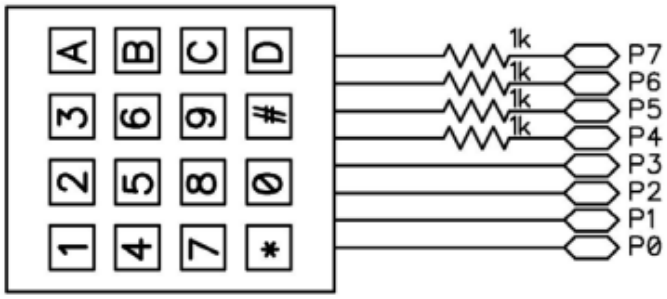


Figure 2.20. Connections Diagrams [13]

2.2.9. 220VDC – 12VDC Step-Down Converte



Figure 2.21. Adapter 12V-2A (Source: Internet)

The power adapter provides a stable 12V DC output with a maximum current of 2A. This output is sufficient to power the entire circuit, including the microcontroller and sensors, ensuring the system operates reliably.

Note: Since the device operates on a 5VDC supply, an integrated power conversion unit has been designed on the main PCB. This unit utilizes the LM7805 IC to step down the 12VDC input from the adapter to a stable 5VDC to power the entire system.

2.2.10. Webcam



Figure 2.22.Webcam C930e (Source: Logitech.com)

The C930e is an advanced 1080p HD webcam that delivers high-resolution video in virtually any environment, including low-light and harshly backlit settings. With H.264 video compression and a generous 90° field of view, the C930e is the premier tool for face-to-face collaboration.

Technical Specifications

- Multiple Resolutions:
  - 1080p/30fps (up to 1920 x 1080 pixels)
  - 720p/30fps (up to 1280 x 720 pixels)
- Camera Megapixels: 3 MP
- Focus Type: Autofocus
- Lens Type: Glass
- Integrated Microphone: Dual omni-directional stereo mics
- Mic Range: Up to 1 meter
- Diagonal Field of View (dFoV): 90°
- Digital Zoom: 4x
- USB Connectivity: USB-A plug-and-play
- RightLight™ 2 Technology: Provides sharp images in various lighting conditions, even in low light.
- Optional Camera Settings App: For controlling pan, tilt, and zoom.

2.3. Actual Wiring Diagram of the System

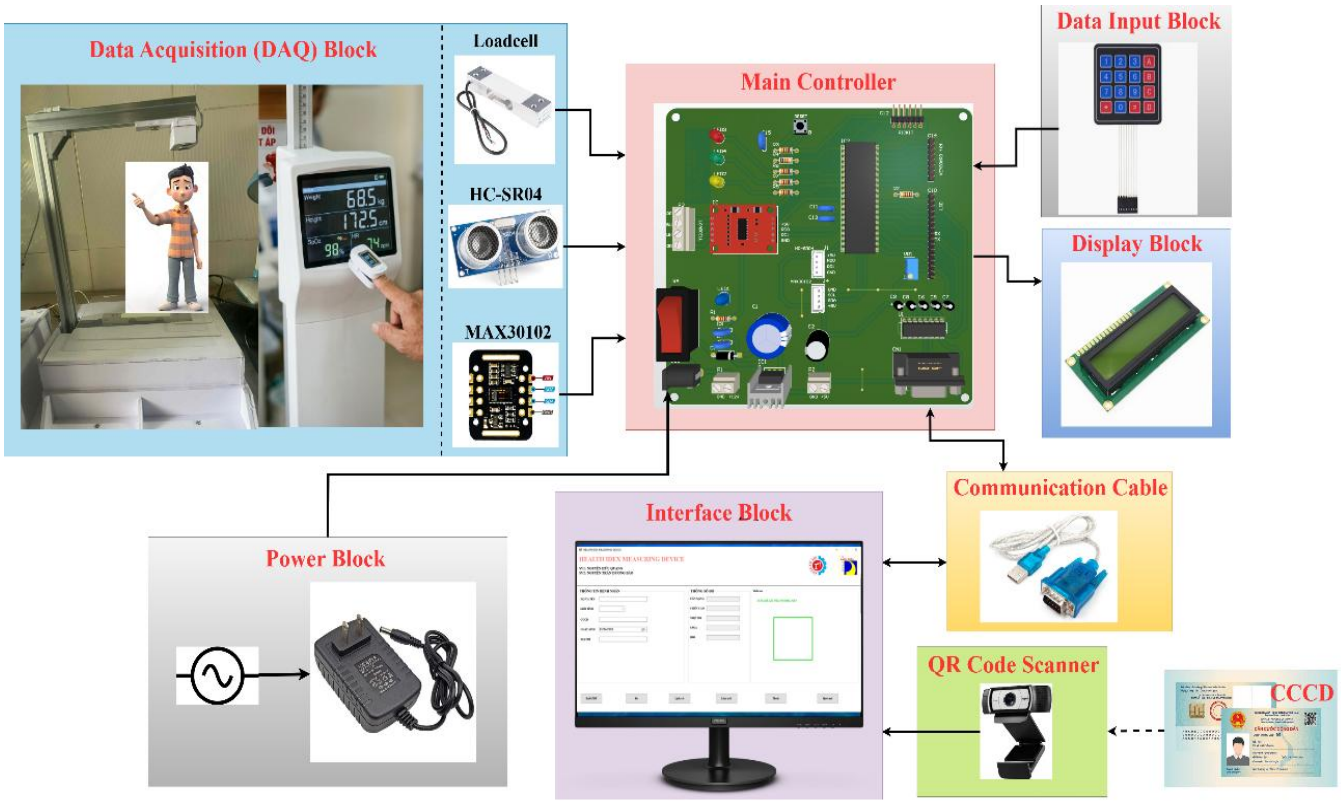


Figure 2.23. System diagram

Block diagram of an integrated multi-sensor data acquisition and processing system. Signals from the Load cell, HC-SR04, and MAX30102 are sent to a central controller for processing, then displayed and transmitted via a computer interface. The system also supports data entry, QR code scanning, and external communication through corresponding functional blocks.

2.4. Altium Design

2.4.1. Introduction



Figure 2.24. Altium Designer 22

Altium Designer is a powerful and globally recognized Electronic Design Automation (EDA) software developed by Altium Limited. Its core distinction lies in the Unified Design philosophy: integrating the entire workflow—from Schematic Capture and PCB Layout to 3D visualization and manufacturing data management—into a single, cohesive environment.

Key Advantages of Altium Designer:

- Real-time Synchronization: Any changes between the schematic and the PCB layout are instantly updated through the Engineering Change Order (ECO) system, ensuring high precision and data integrity.
- Realistic 3D Management: The software provides a realistic 3D engine that allows designers to visualize and check for physical clearances between components and enclosures. This significantly minimizes errors during the final assembly phase.
- Intelligent Routing: Advanced tools such as Interactive Routing and ActiveRoute support high-speed routing and the handling of industrial-standard differential pairs.
- Design Rule Check (DRC): A comprehensive system of design rules ensures that layouts consistently comply with international standards (IPC) and are compatible with the capabilities of modern manufacturing facilities.

2.4.2. Schematic

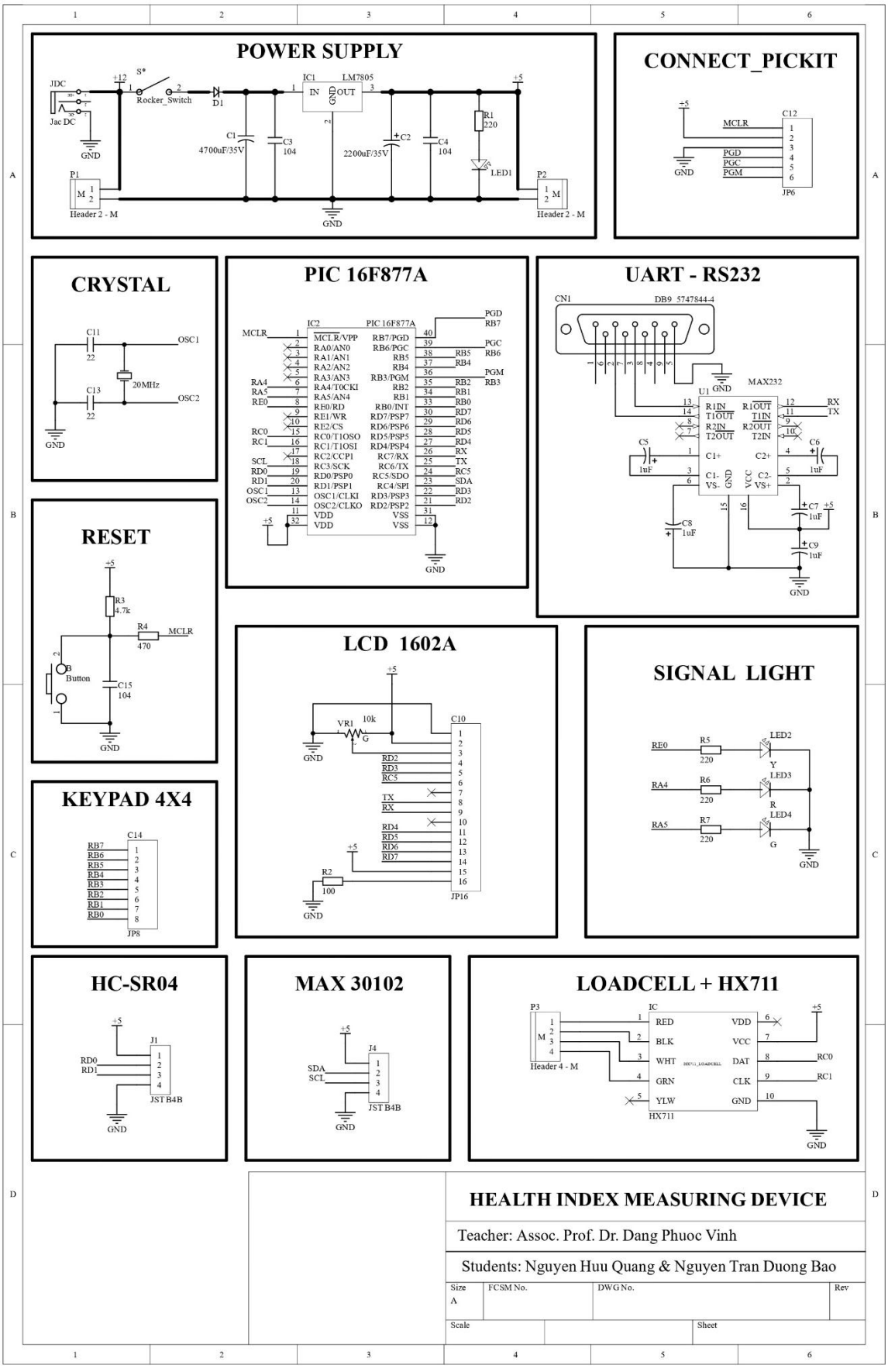


Figure 2.25. Schematic

2.4.3. Layout Design (PCB)

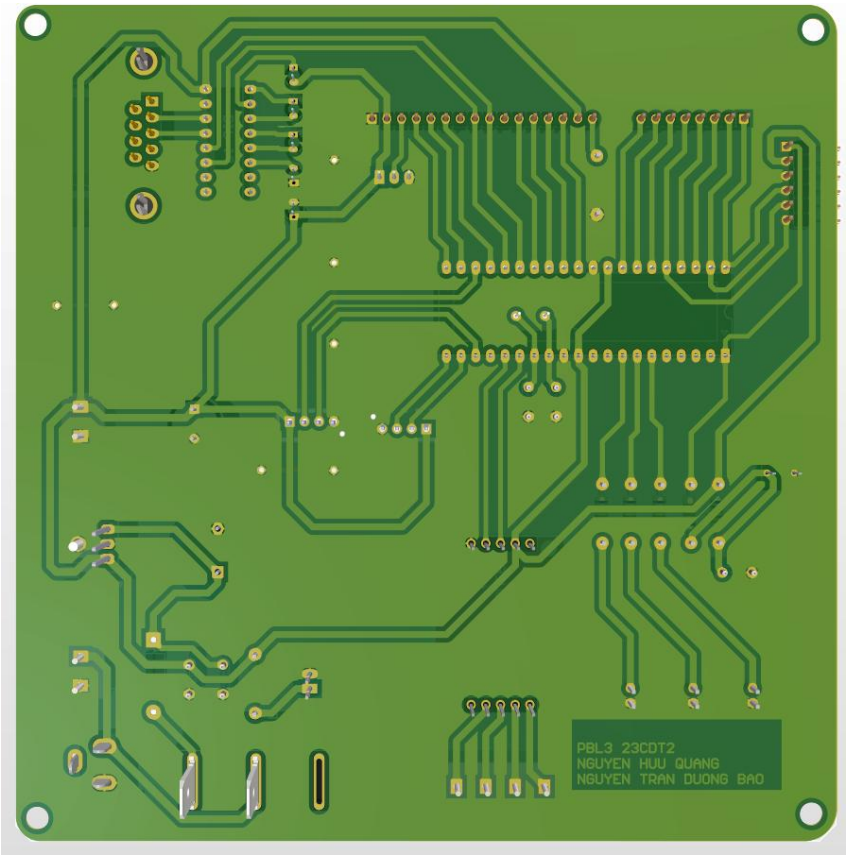


Figure 2.26. 3D PCB Bottom

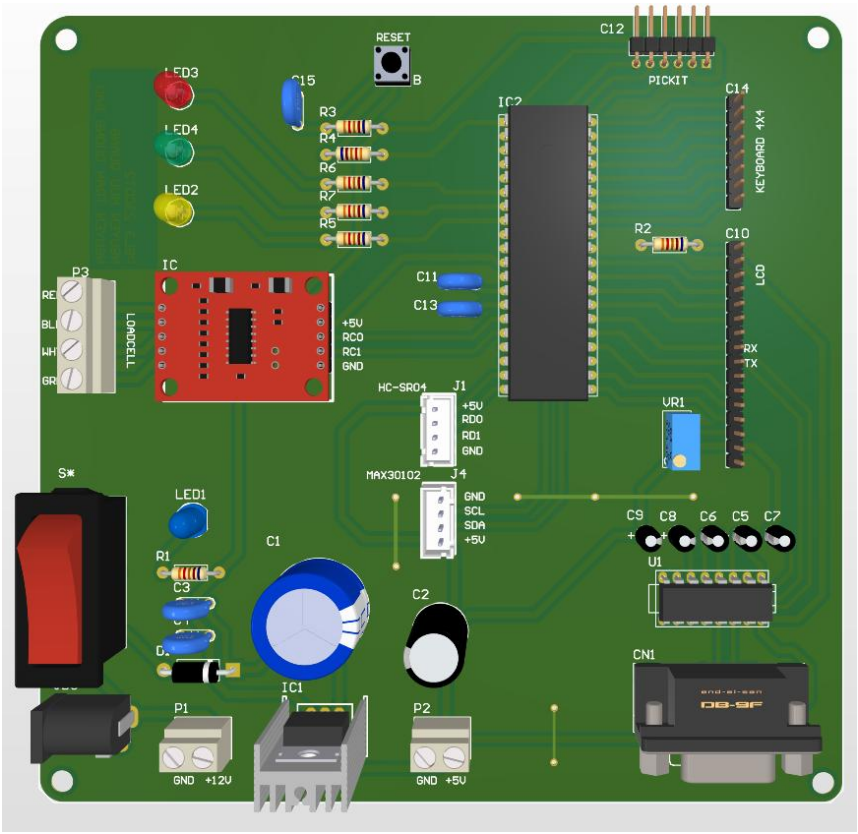


Figure 2.27. 3D PCB Top

CHAPTER 3: SOFTWARE DESIGN

3.1. CCS C Compiler

3.1.1. Introduction to CCS C Compiler



Figure 3.1. CCS C Compiler Logo

The CCS C Compiler is a highly efficient integrated development environment (IDE) specifically designed for programming Microchip's PIC microcontrollers. It provides a user-friendly C programming interface that simplifies the hardware-level complexities, allowing developers to focus on application logic rather than low-level assembl language.

3.1.2. Flowchart

a. Main flowchart

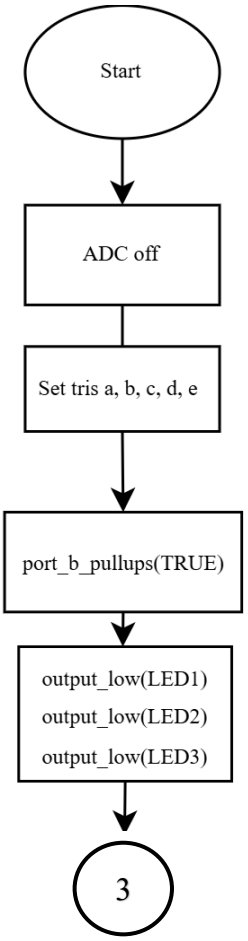


Figure 3.2. Main Flowchart Page1

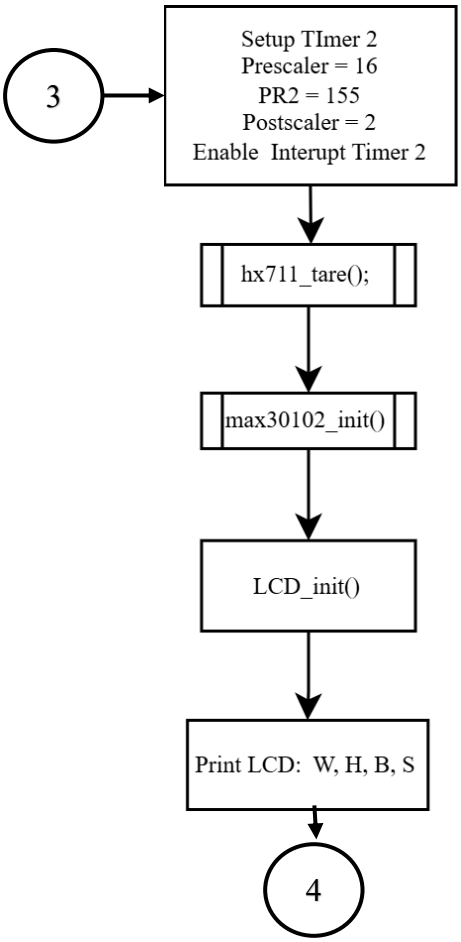


Figure 3.3. Main Flowchart Page2

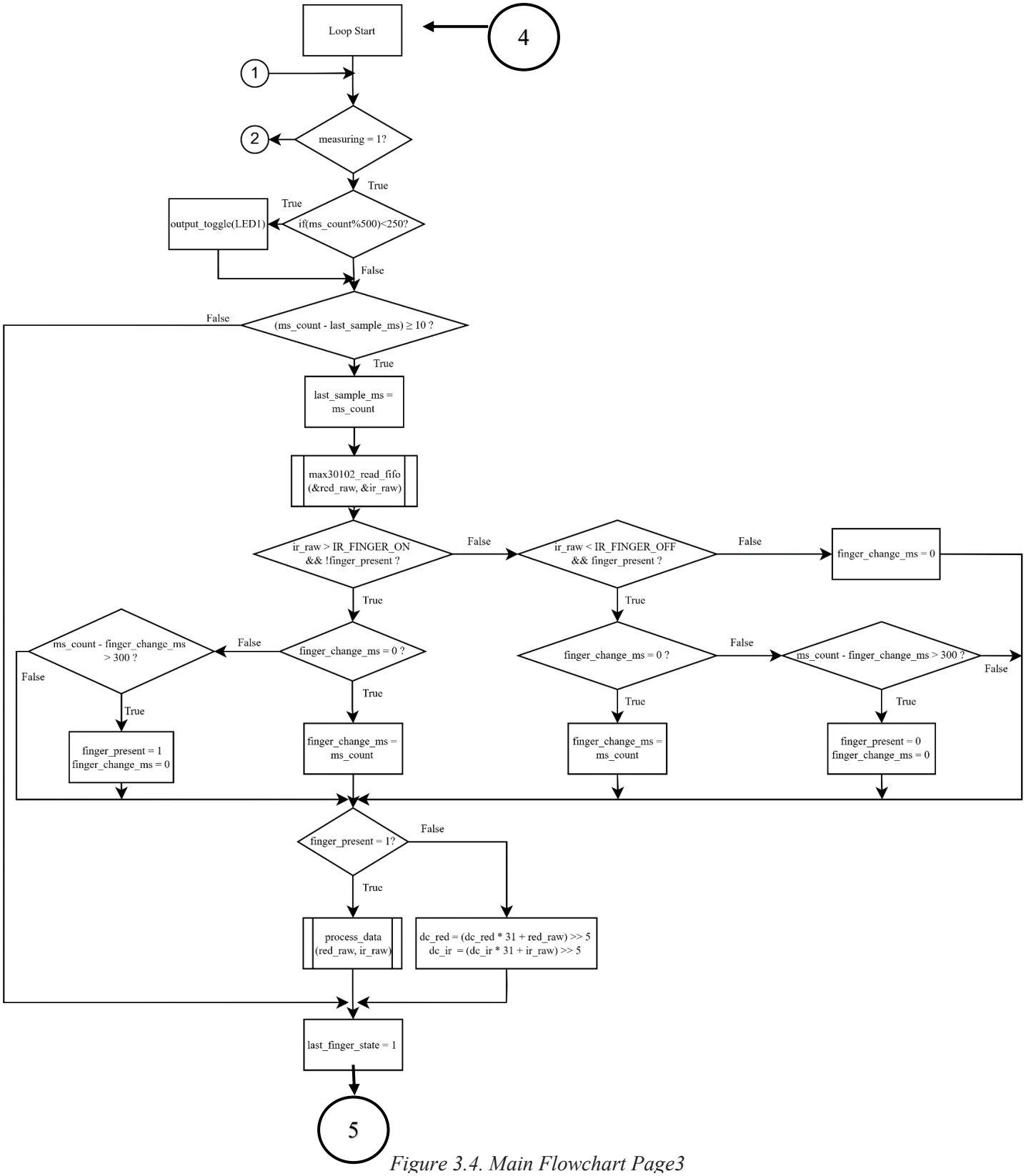


Figure 3.4. Main Flowchart Page3

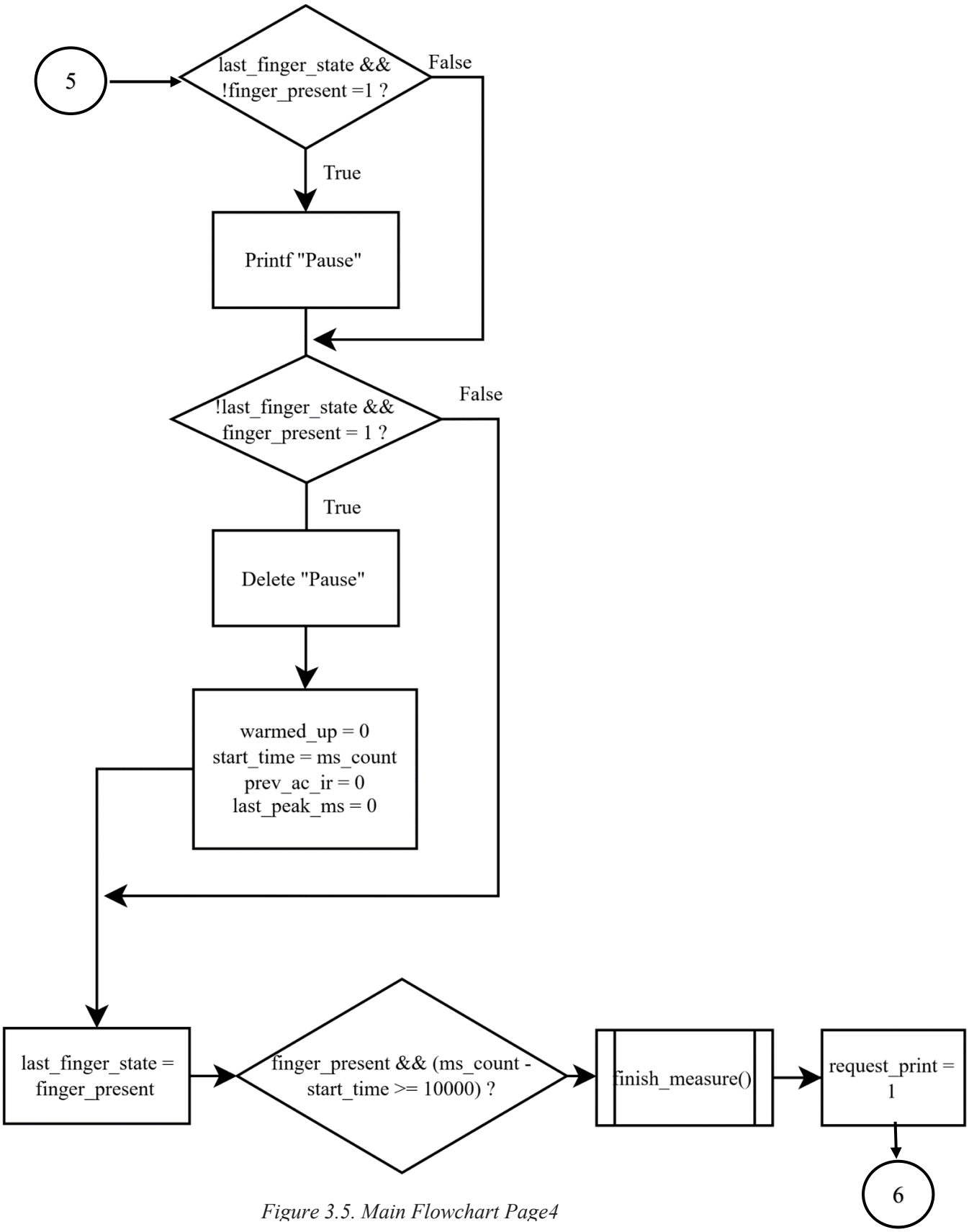


Figure 3.5. Main Flowchart Page4

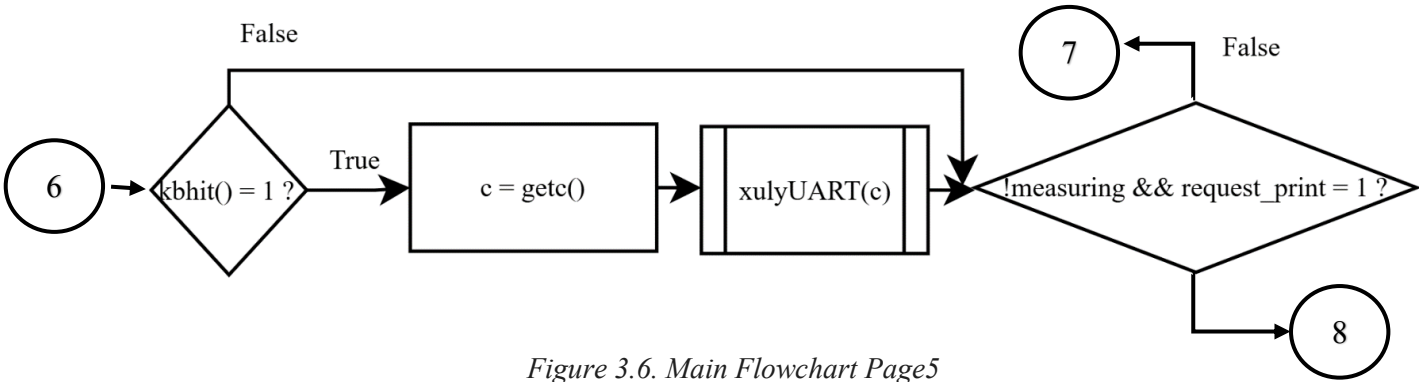


Figure 3.6. Main Flowchart Page5

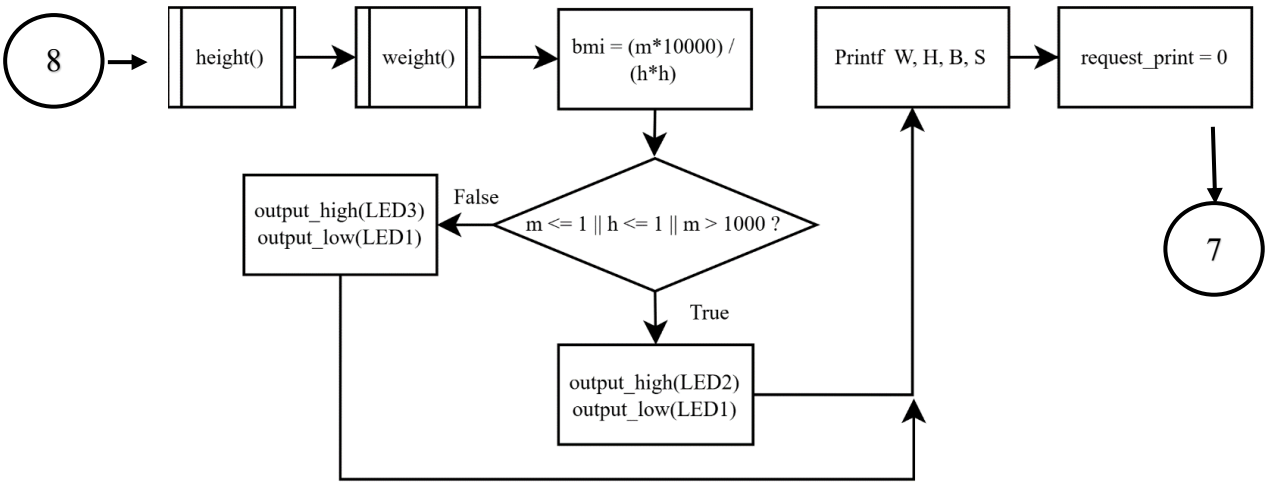


Figure 3.7. Main Flowchart Page6

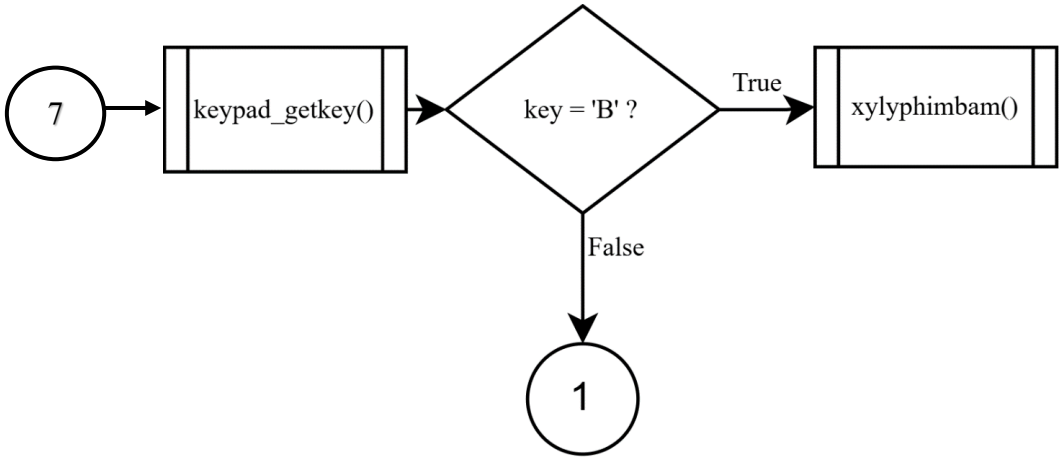


Figure 3.8. Main Flowchart Page7

The main function initializes all hardware components, including sensors, LCD, keypad, timers, and communication interfaces. It then continuously runs a loop to handle user input, acquire sensor data, and control the measurement process. When a measurement is requested, the system collects heart rate, SpO2, weight, and height data, processes the results, and displays them on the LCD as well as sends them through UART. The program ensures real-time operation by managing timing, sensor states, and user interactions.

b. Weight Flowchart

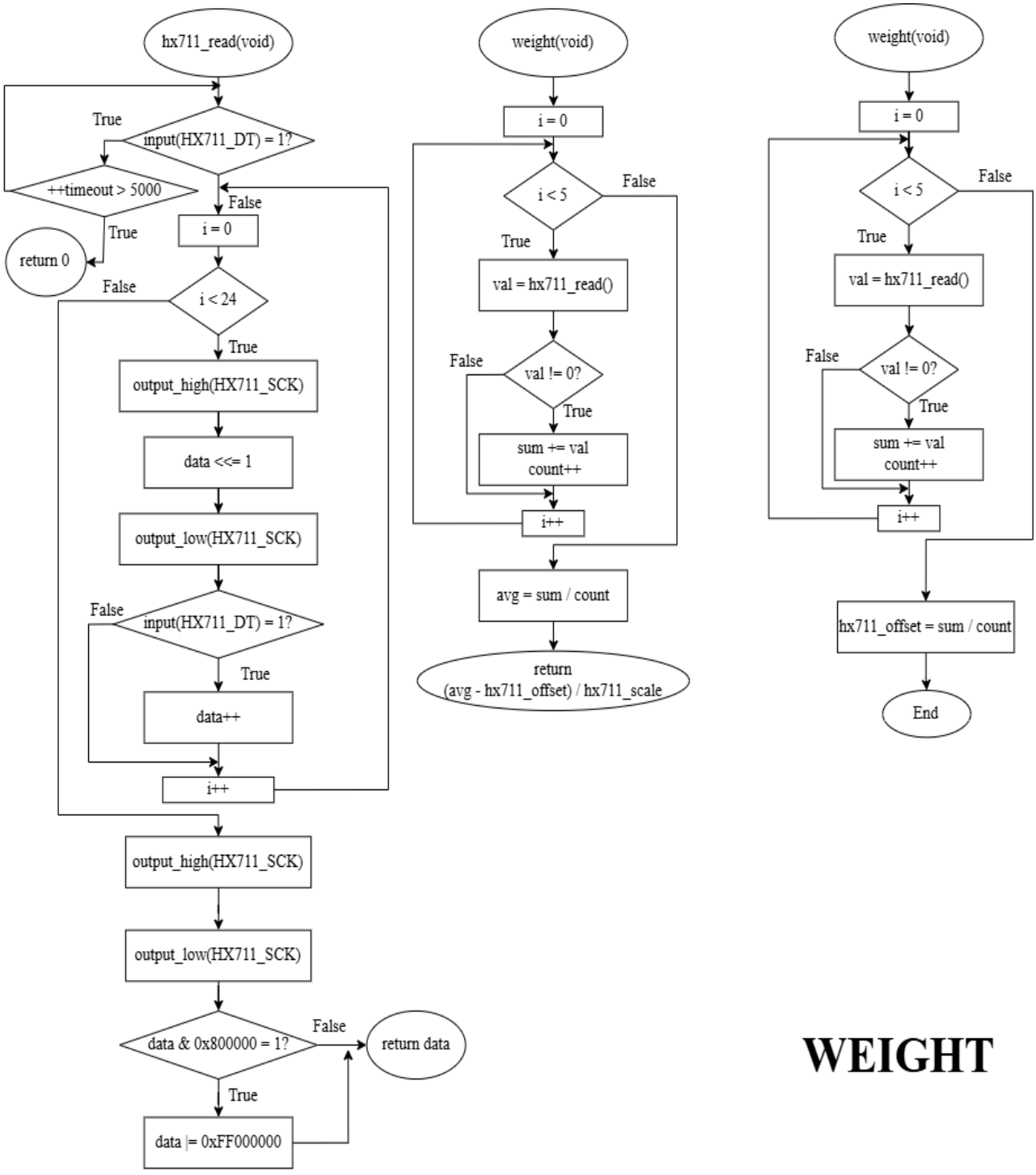


Figure 3.9. Weight Flowchart

These functions are used to measure weight. First, the system performs a tare operation to determine the offset value. During measurement, multiple samples are collected and averaged to reduce noise. The net value is calculated by subtracting the offset and applying a calibration scale factor. Finally, the result is converted into kilograms and displayed as the measured weight.

c. Height Flowchart

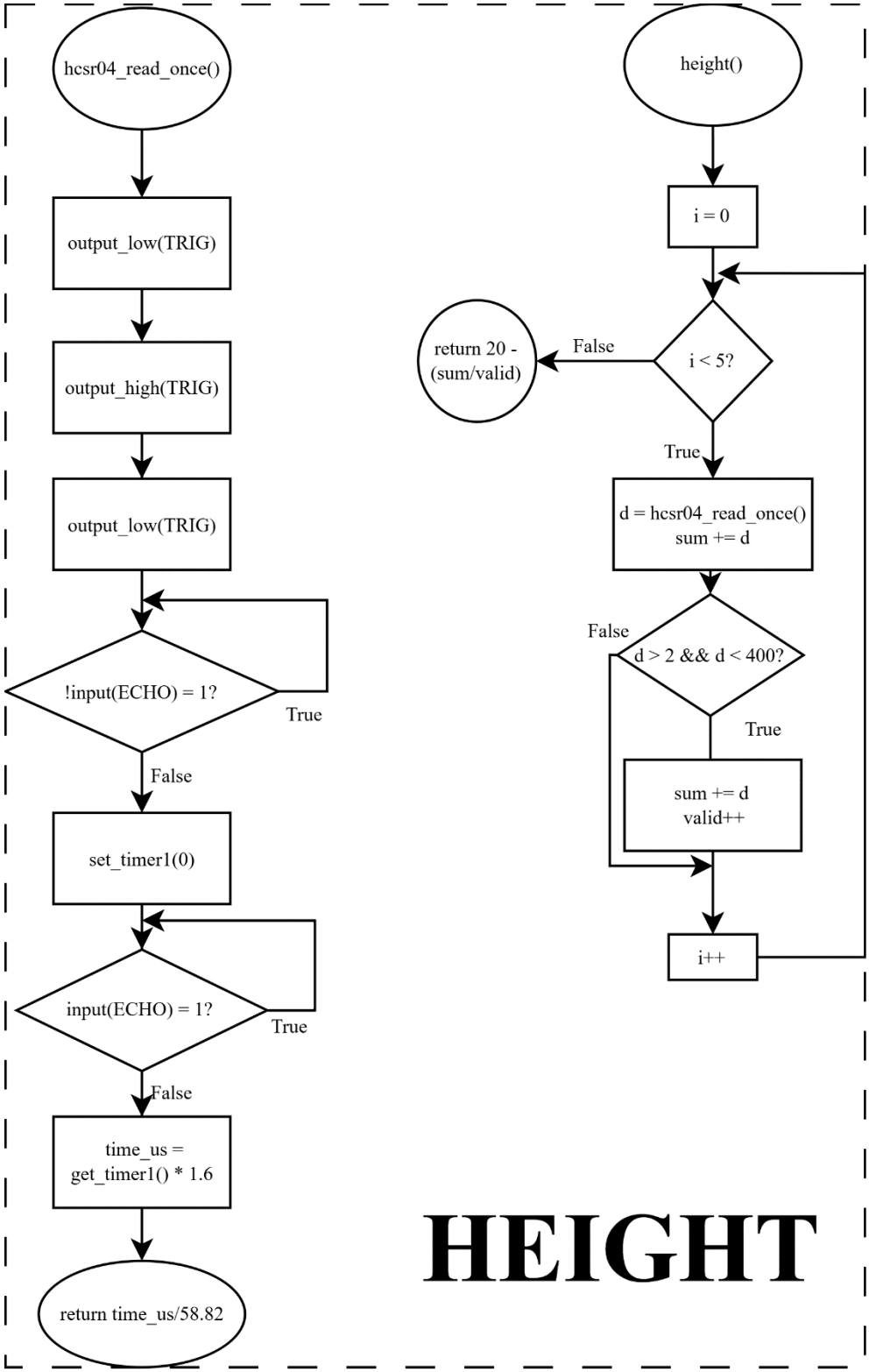
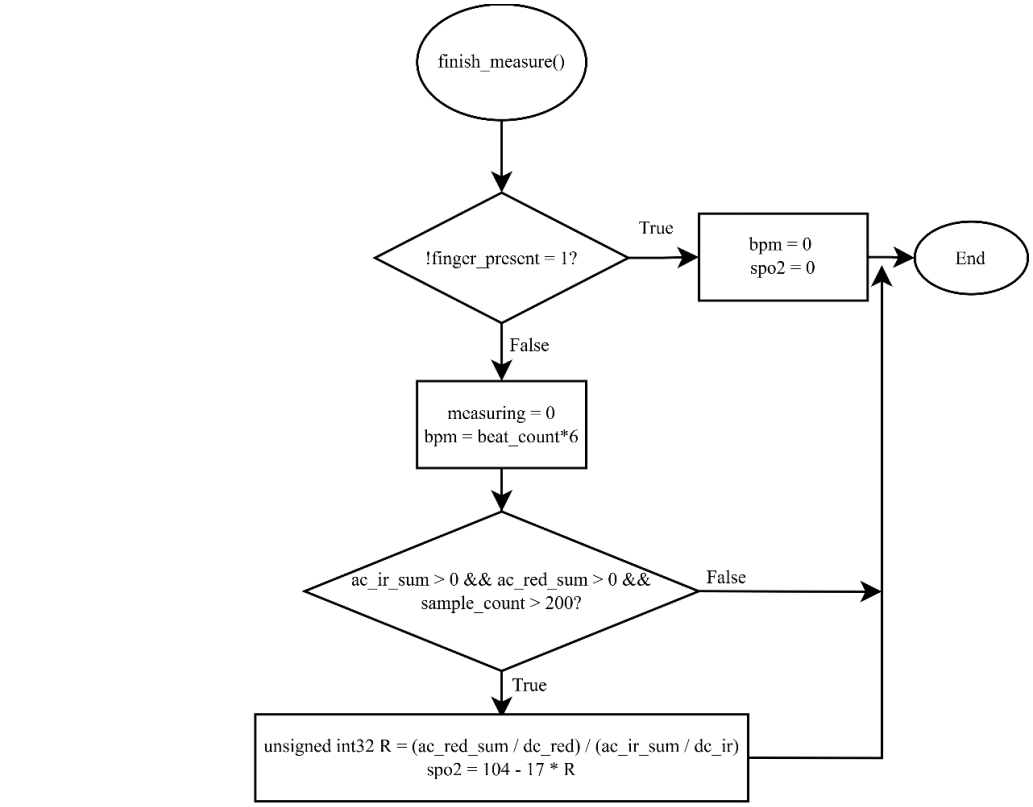


Figure 3.10. Height Flowchart

These functions are used to measure height. The principle is to set the TRIG pin to a high level to generate an ultrasonic pulse, then wait for the ECHO pin to receive the reflected signal. Based on the time delay, the distance from the top of the device to the patient's head is calculated. The final height is obtained by subtracting this distance from the total height of the device.

d. BPM & SPO2 Flowchart



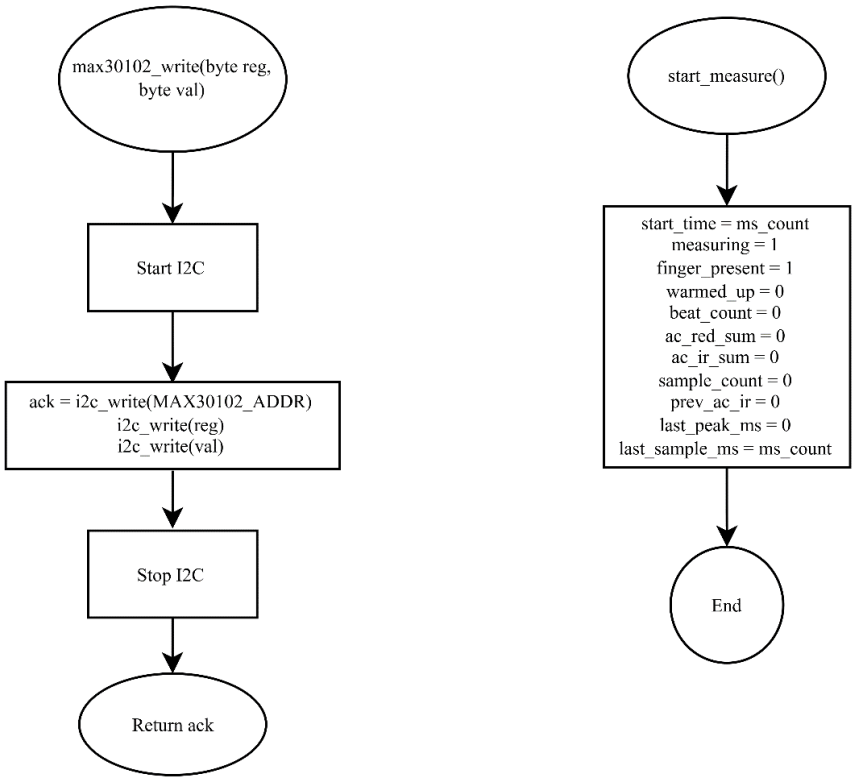


Figure 3.11. BPM & SPO2 Flowchart Page1

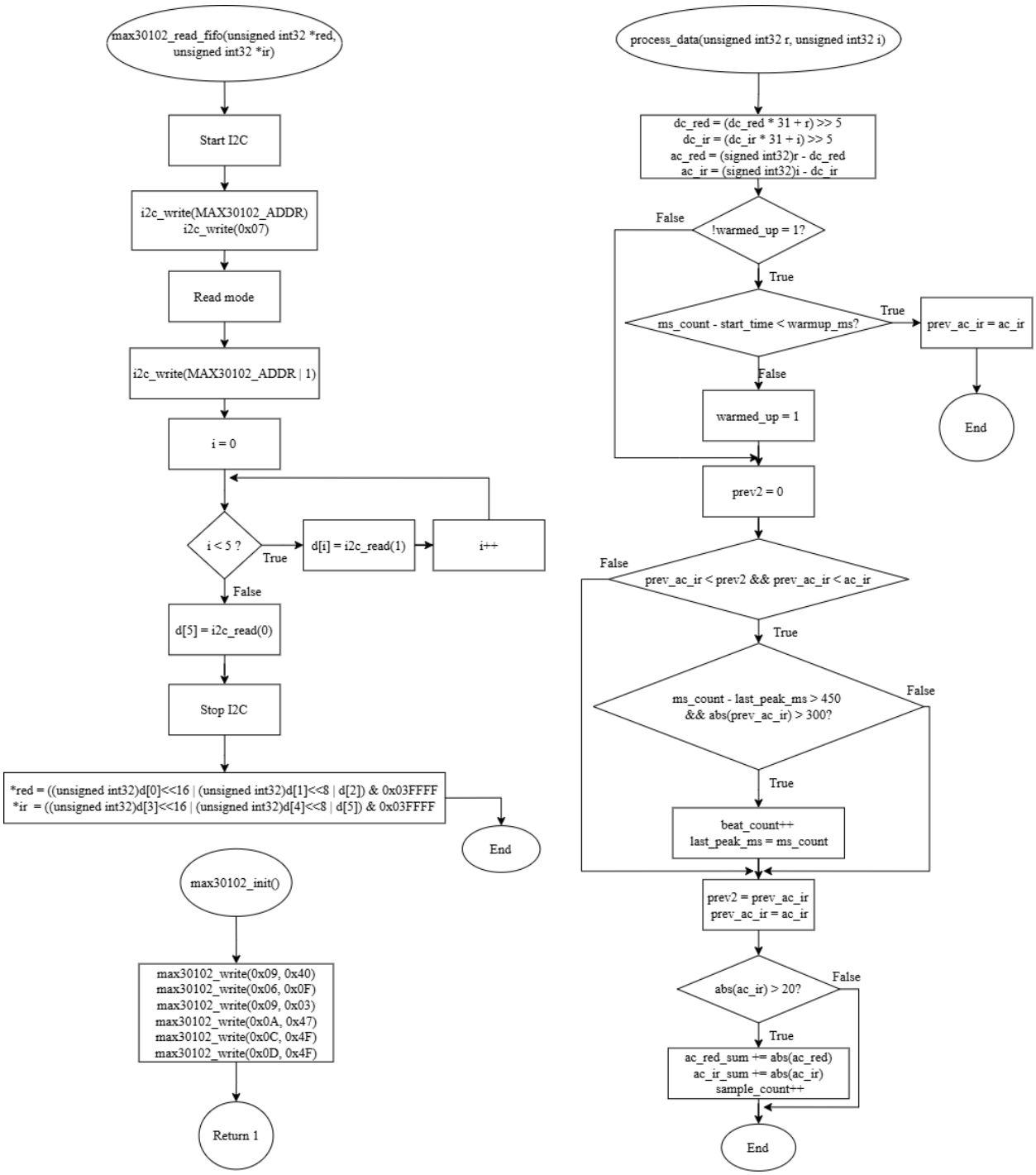


Figure 3.12. BPM & SPO2 Flowchart Page2

These functions are used to measure height. The heart rate measurement algorithm uses the MAX30102 sensor to collect red and infrared signals from the finger. The system separates DC and AC components to remove noise and detect pulse variations. Heartbeats are identified by detecting peaks in the filtered infrared signal over time. The total number of beats is counted and converted into beats per minute (BPM). Additionally, the ratio between red and infrared signals is used to estimate blood oxygen saturation (SpO2).

e. Keypad Flowchart

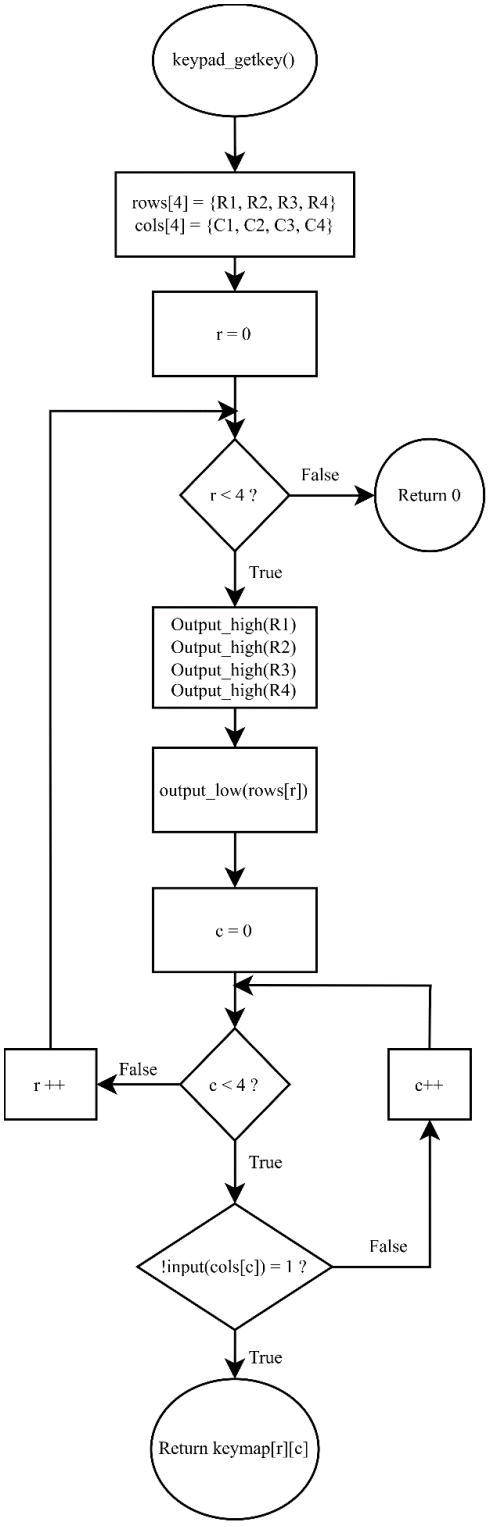


Figure 3.13. Keypad Flowchart

This function implements a software-based matrix keypad control. It scans each row and column by setting rows to low level and reading the column inputs to detect a pressed key.

f. Start Measure Flowchart

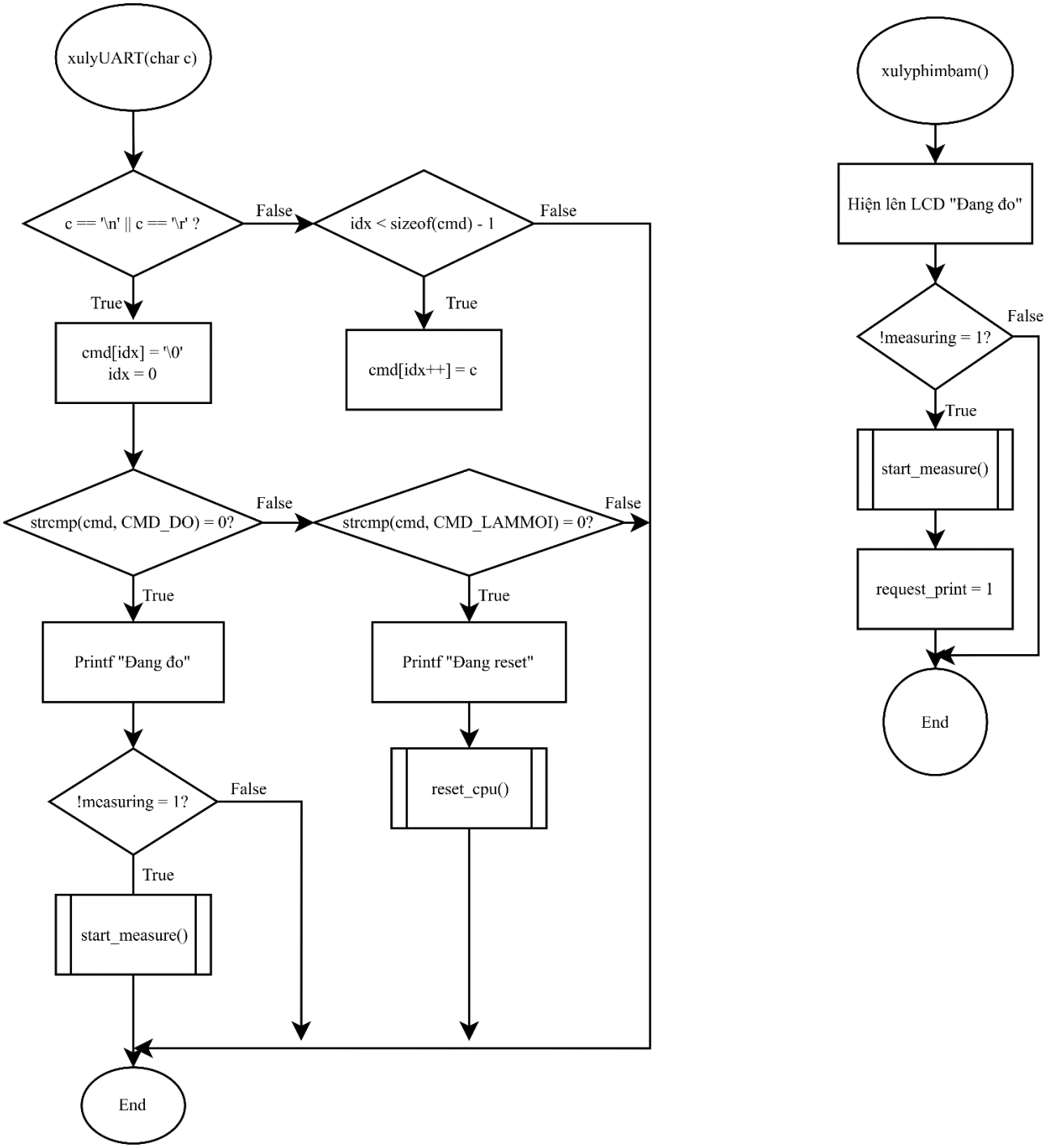


Figure 3.14. Start Measure Flowchart

These two functions handle the execution process. There are two control methods: pressing the physical “B” button on the device, or using the “Đo” and “Làm mới” commands from the software interface.

g. INT TIMER2

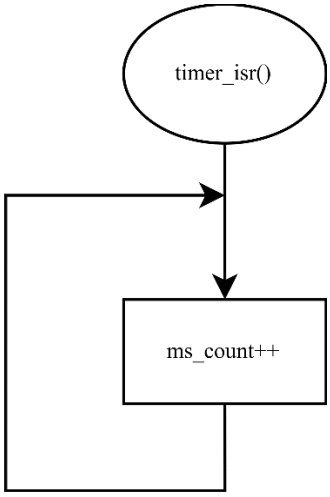


Figure 3.15. INT TIMER2 Flowchart

This is an interrupt service routine triggered when Timer2 overflows. It is used to increment the ms_count variable, which serves as a time base for the system.

3.1.3. Code PBL3_main.c

```
#include <16F877A.h>
#include <string.h>
#include <stdlib.h>
#fuses HS,NOWDT,NOBROWNOUT, NOLVP
#use delay(crystal=20MHz)
#use rs232(baud=9600, xmit=PIN_C6, rcv=PIN_C7, stream=PC_UART)
#define MAX30102_ADDR 0xAE
#use i2c(MASTER, SDA=PIN_C4, SCL=PIN_C3, FAST = 100000, FORCE_HW)
#define LCD_ENABLE_PIN  PIN_C5
#define LCD_RS_PIN      PIN_D2
#define LCD_RW_PIN      PIN_D3
#define LCD_DATA4       PIN_D4
#define LCD_DATA5       PIN_D5
#define LCD_DATA6       PIN_D6
#define LCD_DATA7       PIN_D7
#include <lcd.c>
#define R1 PIN_B0
#define R2 PIN_B1
#define R3 PIN_B2
#define R4 PIN_B3
#define C1 PIN_B4
#define C2 PIN_B5
#define C3 PIN_B6
#define C4 PIN_B7
#define TRIG PIN_D0
#define ECHO PIN_D1
#define HX711_DT   PIN_C0
#define HX711_SCK  PIN_C1
#define LED1 PIN_E0
```

```
#define LED2 PIN_A4
#define LED3 PIN_A5
unsigned int32 ms_count = 0;
unsigned int32 start_time = 0;
int1 measuring = 0;
int1 request_print = 0;
unsigned int32 red_raw, ir_raw;
unsigned int16 beat_count = 0;
unsigned int32 dc_ir = 0, dc_red = 0;
signed int32 ac_ir, ac_red;
signed int32 prev_ac_ir = 0;
float bpm = 0, spo2 = 0;
float ac_red_sum = 0, ac_ir_sum = 0;
unsigned int32 sample_count = 0;
unsigned int32 last_sample_ms = 0;
float h = 0, m = 0;
float bmi = 0;
float hx711_offset = 0;
float hx711_scale = 2852.35f;
char CMD_DO[3] = {'D','o','\0'};
char CMD_LAMMOI[8] = {'L','a','m',' ','m','o','i','\0'};
char key, cmd[10];
unsigned int32 warmup_ms = 2000;
int1 warmed_up = 0;
unsigned int32 last_peak_ms = 0;
unsigned int32 finger_change_ms = 0;
#define REFRACTORY_MS   450
#define IR_FINGER_ON   45000
#define IR_FINGER_OFF  35000
int1 finger_present = 0;
#int_TIMER2
void timer2_isr() {
ms_count++;
}
char keypad_getkey() {
char keymap[4][4] = {
{'1','2','3','A'},
{'4','5','6','B'},
{'7','8','9','C'},
{'*','0','#','D'}
};
int rows[4] = {R1, R2, R3, R4};
int cols[4] = {C1, C2, C3, C4};
int r, c;
for(r = 0; r < 4; r++) {
output_high(R1);
output_high(R2);
output_high(R3);
```

```
78  output_high(R4);
79  output_low(rows[r]);
80  delay_us(10);
81  for(c = 0; c < 4; c++) {
82  if(!input(cols[c])) {
83  delay_ms(20);
84  if(!input(cols[c])) {
85  return keymap[r][c];
86  }
87  }
88  }
89  }
90  return 0;
91  }
92  int1 max30102_write(byte reg, byte val) {
93  int1 ack;
94  i2c_start();
95  ack = i2c_write(MAX30102_ADDR);
96  i2c_write(reg);
97  i2c_write(val);
98  i2c_stop();
99  return ack;
100 }
101 void max30102_read_fifo(unsigned int32 *red, unsigned int32 *ir) {
102 byte d[6];
103 i2c_start();
104 i2c_write(MAX30102_ADDR);
105 i2c_write(0x07);
106 i2c_start();
107 i2c_write(MAX30102_ADDR | 1);
108 for(int i = 0; i < 5; i++)
109 d[i] = i2c_read(1);
110 d[5] = i2c_read(0);
111 i2c_stop();
112 *red = ((unsigned int32)d[0]<<16 | (unsigned int32)d[1]<<8 | d[2]) & 0x03FFFF;
113 *ir  = ((unsigned int32)d[3]<<16 | (unsigned int32)d[4]<<8 | d[5]) & 0x03FFFF;
114 }
115 int1 max30102_init() {
116 max30102_write(0x09, 0x40);
117 delay_ms(100);
118 max30102_write(0x06, 0x0F);
119 max30102_write(0x09, 0x03);
120 max30102_write(0x0A, 0x47);
121 max30102_write(0x0C, 0x4F);
122 max30102_write(0x0D, 0x4F);
123 return 1;
124 }
125 void process_data(unsigned int32 r, unsigned int32 i) {
```

```
126   dc_red = (dc_red * 31 + r) >> 5;
127   dc_ir = (dc_ir * 31 + i) >> 5;
128   ac_red = (signed int32)r - dc_red;
129   ac_ir = (signed int32)i - dc_ir;
130   if(!warmed_up) {
131   if(ms_count - start_time < warmup_ms) {
132   prev_ac_ir = ac_ir;
133   return;
134   } else {
135   warmed_up = 1;
136   }
137   }
138   static signed int32 prev2 = 0;
139   if(prev_ac_ir < prev2 && prev_ac_ir < ac_ir) {
140   if(ms_count - last_peak_ms > REFRACTORY_MS && abs(prev_ac_ir) > 300) {
141   beat_count++;
142   last_peak_ms = ms_count;
143   }
144   }
145   prev2 = prev_ac_ir;
146   prev_ac_ir = ac_ir;
147   if(abs(ac_ir) > 20) {
148   ac_red_sum += (float)abs(ac_red);
149   ac_ir_sum += (float)abs(ac_ir);
150   sample_count++;
151   }
152   }
153   void start_measure() {
154   start_time = ms_count;
155   measuring = 1;
156   finger_present = 1;
157   warmed_up = 0;
158   beat_count = 0;
159   ac_red_sum = 0;
160   ac_ir_sum = 0;
161   sample_count = 0;
162   prev_ac_ir = 0;
163   last_peak_ms = 0;
164   last_sample_ms = ms_count;
165   }
166   void finish_measure() {
167   if(!finger_present) {
168   bpm = 0;
169   spo2 = 0;
170   return;
171   }
172   measuring = 0;
173   bpm = (float)beat_count * 6.0f;
```

```
174   if(ac_ir_sum > 0 && ac_red_sum > 0 && sample_count > 200) {
175   float R = (ac_red_sum / dc_red) / (ac_ir_sum / dc_ir);
176   spo2 = 104.0 - 17.0 * R;
177   if(spo2 > 100) spo2 = 100;
178   if(spo2 < 85)  spo2 = 85;
179   } else {
180   bpm = 0;
181   spo2 = 0;
182   }
183   }
184   signed int32 hx711_read(void) {
185   signed int32 data = 0;
186   int8 i;
187   unsigned int16 timeout = 0;
188   while(input(HX711_DT)) {
189   delay_us(10);
190   if(++timeout > 5000) return 0;
191   }
192   for(i = 0; i < 24; i++) {
193   output_high(HX711_SCK);
194   delay_us(1);
195   data <<= 1;
196   output_low(HX711_SCK);
197   delay_us(1);
198   if(input(HX711_DT)) data++;
199   }
200   output_high(HX711_SCK);
201   delay_us(1);
202   output_low(HX711_SCK);
203   if(data & 0x800000)
204   data |= 0xFF000000;
205   return data;
206   }
207   void hx711_tare(void) {
208   signed int32 val;
209   float sum = 0;
210   int count = 0;
211   for(int i = 0; i < 5; i++) {
212   val = hx711_read();
213   if(val != 0) {
214   sum += (float)val;
215   count++;
216   }
217   delay_ms(10);
218   }
219   if(count == 0) hx711_offset = 0;
220   hx711_offset = sum / count;
221   }
```

```
float weight(void) {
signed int32 val;
float sum = 0;
int count = 0;
for(int i = 0; i < 5; i++) {
val = hx711_read();
if(val != 0) {
sum += (float)val;
count++;
}
delay_ms(10);
}
if(count == 0) return 0;
float avg = sum / count;
float net = (avg - hx711_offset) / hx711_scale;
if(net < 0) net = 0;
return net/1000;
}
float hcsr04_read_once() {
unsigned int16 timeout;
unsigned int32 time_us;
output_low(TRIG);
delay_us(2);
output_high(TRIG);
delay_us(10);
output_low(TRIG);
timeout = 0;
while(!input(ECHO)) {
delay_us(1);
if(++timeout > 30000) return 0;
}
set_timer1(0);
timeout = 0;
while(input(ECHO)) {
delay_us(1);
if(++timeout > 30000) return 0;
}
time_us = get_timer1() * 1.6;
return (float)time_us / 58.82;
}
float height() {
float sum = 0;
int valid = 0;
for(int i = 0; i < 5; i++) {
float d = hcsr04_read_once();
if(d > 2 && d < 400) {
sum += d;
valid++;
```

```
}
delay_ms(10);
}
if(valid == 0) return 0;
float hei = 20 - (sum / valid);
if (hei < 0) hei = 0;
return hei;
}
void xulyUART(char c) {
static int idx = 0;
if(c == '\n' || c == '\r') {
cmd[idx] = '\0';
idx = 0;
if(strcmp(cmd, CMD_DO) == 0) {
lcd_putc('\f');
lcd_gotoxy(1,1);
printf(lcd_putc, "Dang do...");
if(!measuring) start_measure();
}
else if(strcmp(cmd, CMD_LAMMOI) == 0) {
lcd_putc('\f');
lcd_gotoxy(1,1);
printf(lcd_putc, "Dang reset...");
reset_cpu();
}
} else {
if(idx < sizeof(cmd) - 1)
cmd[idx++] = c;
}
}
void xulyphimbam() {
lcd_putc('\f');
lcd_gotoxy(1,1);
printf(lcd_putc, "Dang do...");
if(!measuring) {
start_measure();
request_print = 1;
}
}
void main() {
setup_adc_ports(NO_ANALOGS);
setup_adc(ADC_OFF);
set_tris_a(0);
set_tris_b(0xF0);
set_tris_c(0x91);
set_tris_d(0x02);
set_tris_e(0x00);
port_b_pullups(TRUE);
```

```
318   output_low(LED1);
319   output_low(LED2);
320   output_low(LED3);
321   setup_timer_1(T1_INTERNAL | T1_DIV_BY_8);
322   setup_timer_2(T2_DIV_BY_16, 155, 2);
323   enable_interrupts(INT_TIMER2);
324   enable_interrupts(GLOBAL);
325   delay_ms(2000);
326   hx711_tare();
327   max30102_init();
328   lcd_init();
329   delay_ms(200);
330   lcd_putc('\f');
331   lcd_gotoxy(1,1);
332   printf(lcd_putc,"W:%5.1f H:%5.1f", m, h);
333   lcd_gotoxy(1,2);
334   printf(lcd_putc,"B:%3.0f S:%3.0f%%", bpm, spo2);
335   while(TRUE) {
336   if(measuring) {
337   if((ms_count % 500) < 250)
338   output_high(LED1);
339   else
340   output_low(LED1);
341   output_low(LED3);
342   output_low(LED2);
343   if(ms_count - last_sample_ms >= 10) {
344   last_sample_ms = ms_count;
345   max30102_read_fifo(&red_raw, &ir_raw);
346   if(ir_raw > IR_FINGER_ON && !finger_present) {
347   if(finger_change_ms == 0) finger_change_ms = ms_count;
348   else if(ms_count - finger_change_ms > 300) {
349   finger_present = 1;
350   finger_change_ms = 0;
351   }
352   }
353   else if(ir_raw < IR_FINGER_OFF && finger_present) {
354   if(finger_change_ms == 0) finger_change_ms = ms_count;
355   else if(ms_count - finger_change_ms > 300) {
356   finger_present = 0;
357   finger_change_ms = 0;
358   }
359   }
360   else {
361   finger_change_ms = 0;
362   }
363   if(finger_present)
364   process_data(red_raw, ir_raw);
365   else {
```

```
366   dc_red = (dc_red * 31 + red_raw) >> 5;
367   dc_ir  = (dc_ir * 31 + ir_raw) >> 5;
368   }
369   }
370   static int1 last_finger_state = 1;
371   if(last_finger_state && !finger_present) {
372   lcd_gotoxy(13,1);
373   printf(lcd_putc,"PAUSE");
374   }
375   if(!last_finger_state && finger_present) {
376   lcd_gotoxy(13,1);
377   printf(lcd_putc,"     ");
378   warmed_up = 0;
379   start_time = ms_count;
380   prev_ac_ir = 0;
381   last_peak_ms = 0;
382   }
383   last_finger_state = finger_present;
384
385   if(finger_present && (ms_count - start_time >= 10000)) {
386   finish_measure();
387   request_print = 1;
388   }
389   }
390   if(kbhit()) {
391   char c = getc();
392   xulyUART(c);
393   }
394   if(!measuring && request_print) {
395   h = height();
396   m = weight();
397   output_low(LED1);
398   if(m <= 0.01 || h <= 0.01 || m > 1)
399   {
400   output_high(LED2);
401   bmi = 0; // không tính
402   }
403   else
404   {
405   output_high(LED3);
406   float hm = h / 100.0;
407   bmi = m / (hm * hm);
408   }
409   lcd_gotoxy(1,1);
410   printf(lcd_putc,"W:%5.3f H:%5.1f", m, h);
411   lcd_gotoxy(1,2);
412   printf(lcd_putc,"B:%3.0f S:%3.0f%%", bpm, spo2);
413   printf("W: %5.1f\r\n", m);
```

```
414     delay_ms(50);
415     printf("H: %5.1f\r\n", h);
416     delay_ms(50);
417     printf("B: %3.0f\r\n", bpm);
418     delay_ms(50);
419     printf("S: %3.0f\r\n", spo2);
420     delay_ms(50);
421     printf("BMI: %5.1f\r\n", bmi);
422     request_print = 0;
423     }
424     key = keypad_getkey();
425     if(key == 'B')
426     xulyphimbam();}}
```

3.2. Visual Studio

3.2.1. Introdution to Visual Studio



Figure 3.16. Visual Studio

Visual Studio is a powerful Integrated Development Environment (IDE) from Microsoft used to develop the system's desktop application. Using the C# language and Windows Forms, it provides an intuitive interface for monitoring health data in real-time. The software efficiently handles Serial Port communication to receive data from the PIC microcontroller and utilizes libraries like iText7 to generate professional PDF medical reports. Its robust debugging tools and seamless hardware integration ensure that patient information is processed accurately and stored securely in the hospital database.

3.2.2. User Interface

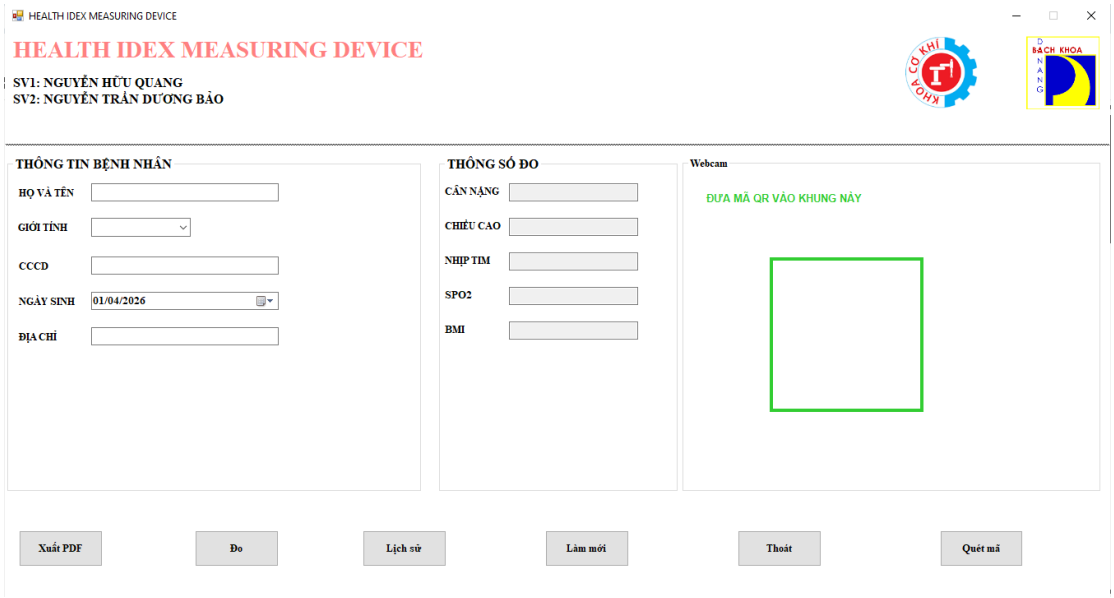


Figure 3.17. User Interface

3.2.3. Code

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.IO;
using System.IO.Ports;
using System.Windows.Forms;
using iText.Kernel.Pdf;
using iText.Layout;
using iText.Layout.Element;
using AForge.Video;
using AForge.Video.DirectShow;
using ZXing;

namespace BVBK {
    public partial class frmtrangchu : Form {
        private FilterInfoCollection captureDevices;
        private VideoCaptureDevice videoSource;
        private frmlichsu formLichSu = null;

        public frmtrangchu() { InitializeComponent(); }

        private void Form1_Load(object sender, EventArgs e) {
            SetupWebcam();
            string[] ports = SerialPort.GetPortNames();
            if (ports.Length > 0) {
                serialPort1.PortName = ports[0];
                serialPort1.BaudRate = 9600;
                if (!serialPort1.IsOpen) serialPort1.Open();
            }
        }

        // --- XỬ LÝ DỮ LIỆU TỪ VI ĐIỀU KHIỂN (UART) ---
        private void serialPort1_DataReceived(object sender,
SerialDataReceivedEventArgs e) {
            string data = serialPort1.ReadLine().Trim();
            this.BeginInvoke(new MethodInvoker(() => {
                if (data.StartsWith("H:")) txtchieucao.Text =
data.Split(':')[1];
                else if (data.StartsWith("W:")) txtcannang.Text =
data.Split(':')[1];
                else if (data.StartsWith("B:")) txtnhiptim.Text =
data.Split(':')[1];
                else if (data.StartsWith("S:")) txtspo2.Text =
data.Split(':')[1];
                else if (data.StartsWith("BMI:")) txtbmi.Text =
data.Split(':')[1];
            }));
        }

        // --- XỬ LÝ QUÉT MÃ QR (CCCD) ---
```

```
45          private void SetupWebcam() {
                captureDevices = new
46  FilterInfoCollection(FilterCategory.VideoInputDevice);
47              if (captureDevices.Count > 0) {
                    videoSource = new
48  VideoCaptureDevice(captureDevices[0].MonikerString);
49                  videoSource.NewFrame += VideoSource_NewFrame;
50              }
51          }
52
                private void VideoSource_NewFrame(object sender,
53  NewFrameEventArgs eventArgs) {
54              Bitmap bitmap = (Bitmap)eventArgs.Frame.Clone();
55              this.Invoke(new MethodInvoker(() => {
56                  pictureBoxWebcam.Image = (Bitmap)bitmap.Clone();
57              }));
58
                BarcodeReader reader = new BarcodeReader { AutoRotate =
59  true, TryInverted = true };
60              var result = reader.Decode(bitmap);
61              if (result != null) {
62                  videoSource.SignalToStop();
                    this.Invoke(new MethodInvoker(() =>
63  ParseCCCD(result.Text)));
64              }
65              bitmap.Dispose();
66          }
67
68          private void ParseCCCD(string data) {
69              string[] parts = data.Split('|');
70              if (parts.Length >= 6) {
71                  txtcccd.Text = parts[0];
72                  txthovaten.Text = parts[2];
73                  cboGioiTinh.Text = parts[4];
74                  txtdiachi.Text = parts[5];
75                  MessageBox.Show("Quét CCCD thành công!");
76              }
77          }
78
79          // --- XUẤT BÁO CÁO PDF (ITEXT7) ---
80          public void XuatFilePDF() {
                string path =
    Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.Deskto
81  p), txthovaten.Text + ".pdf");
82              using (PdfWriter writer = new PdfWriter(path))
83              using (PdfDocument pdf = new PdfDocument(writer)) {
84                  Document doc = new Document(pdf);
                    doc.Add(new Paragraph("BỆNH VIỆN BÁCH
85  KHOA").SetBold());
86                  doc.Add(new Paragraph("HỌ TÊN: " + txthovaten.Text));
87                  doc.Add(new Paragraph("CCCD: " + txtcccd.Text));
```

```
                doc.Add(new Paragraph("SPO2: " + txtspo2.Text + "% |
88  NHỊP TIM: " + txtnhiptim.Text + "bpm"));
89              doc.Add(new Paragraph("BMI: " + txtbmi.Text));
                doc.Add(new Paragraph("NGÀY KHÁM: " +
90  DateTime.Now.ToString("dd/MM/yyyy HH:mm")));
91              doc.Close();
92          }
93          System.Diagnostics.Process.Start(path);
94      }
95
96      // --- ĐIỀU KHIỂN NÚT BẤM ---
97      private void btndo_Click(object sender, EventArgs e) {
98          if (serialPort1.IsOpen) serialPort1.Write("Do\n");
99      }
100
101     private void btnlammoi_Click(object sender, EventArgs e) {
            foreach (Control c in this.Controls) if (c is TextBox)
102 c.Text = "";
103         if (serialPort1.IsOpen) serialPort1.Write("Lam moi\n");
104     }
105
106     private void btnxuatpdf_Click(object sender, EventArgs e) {
107         XuatFilePDF();
108         // Cập nhật vào danh sách lịch sử
            if (formLichSu == null || formLichSu.IsDisposed) formLichSu
109 = new frmlichsu();
            formLichSu.ThemVaoListView("1", txthovaten.Text,
    cboGioiTinh.Text, txtcccd.Text, "", txtdiachi.Text, txtcannang.Text,
    txtchieucao.Text, txtnhiptim.Text, txtspo2.Text, txtbmi.Text,
110 DateTime.Now.ToString());
111     }
112 }
113 }
```

CHAPTER 4: DISCUSSION AND CONCLUSION

4.1. System Prototype

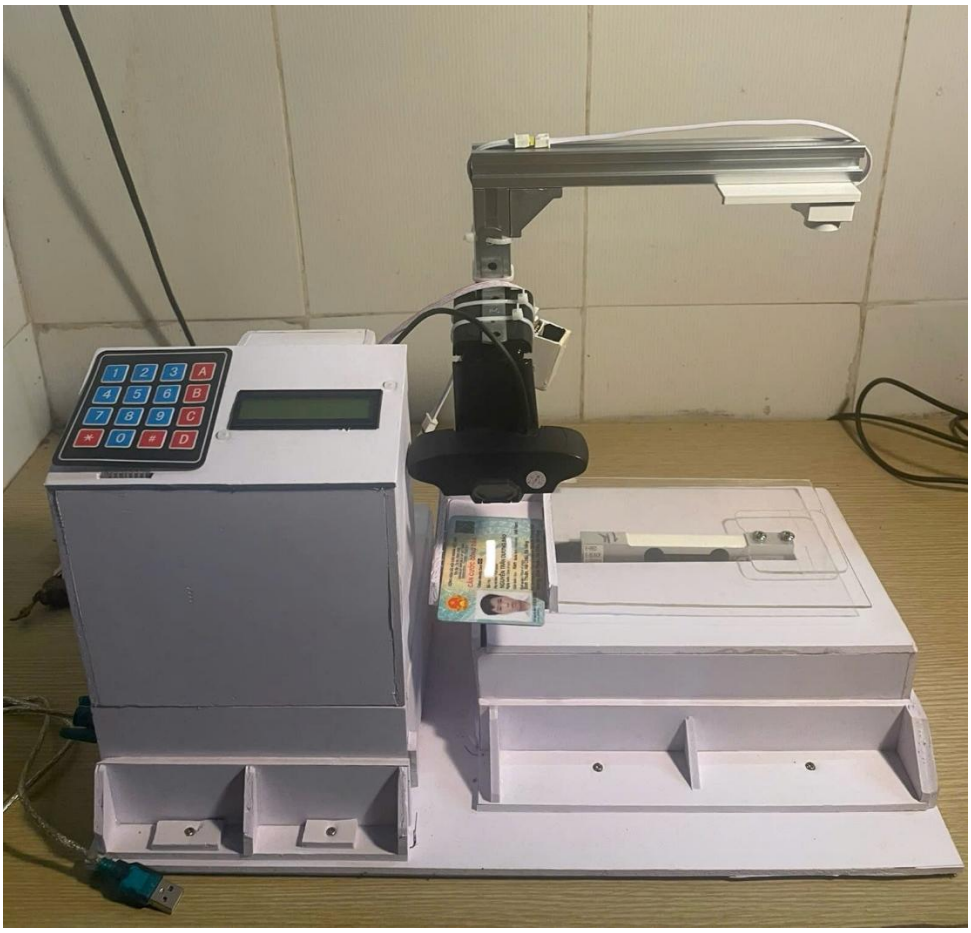


Figure 4.1. Final Prototype

4.2. User Guide

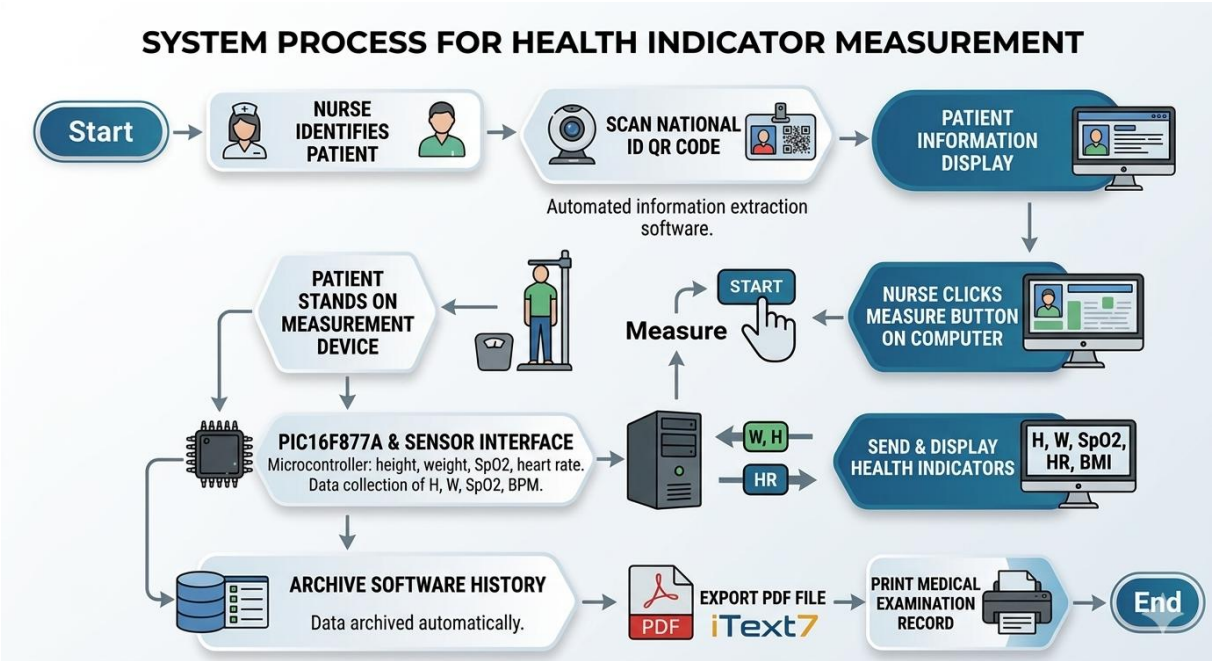


Figure 4.2. Operational Flowchart

4.2.1. Step 1: System Initialization and Software Launch

First, connect both the Health Idex Measuring Device and the external Webcam to the computer via available USB ports.

On the computer, locate the **Health Idex Measuring Device.exe** application and double-click to launch the program.

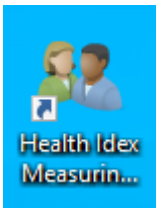


Figure 4.3. Appication Icon

Upon starting, the software will automatically scan for available COM ports; a notification message saying **" Đã kết nối COM "** will appear once a successful link with the PIC16F877A microcontroller is established.

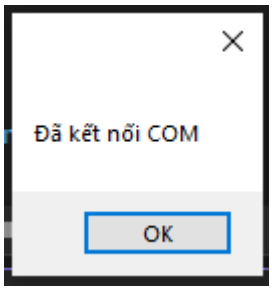


Figure 4.4. Notification message

4.2.2. Step2: QR Code Scanning and Data Entry

The next step involves automated patient registration. The nurse positions the patient's ID card (CCCD) in front of the webcam, ensuring the QR code is aligned within the green target frame on the dashboard.



Figure 4.5. Green target frame

Using the ZXing library, the system decodes the QR string and automatically populates the patient's personal details including Full Name, ID Number, Date of Birth, and Address into the respective text fields. A confirmation box "Đã quét thành công thông tin CCCD!" will appear once the data is correctly parsed.

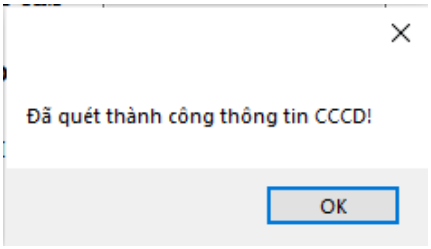


Figure 4.6. Notification message

4.2.3. Step 3: Vital Signs Measurement and BMI Calculation

After the information is entered, the nurse assists the patient to stand steadily on the measuring scale and ensures the pulse sensor is properly attached to their finger. By clicking the "Đo" button on the interface, the software triggers the PIC16F877A to begin data acquisition. Within a few seconds, real-time values for Height, Weight, SpO2, and Heart Rate are transmitted to the PC via the Serial Port. The system then automatically calculates the BMI index based on these measurements.

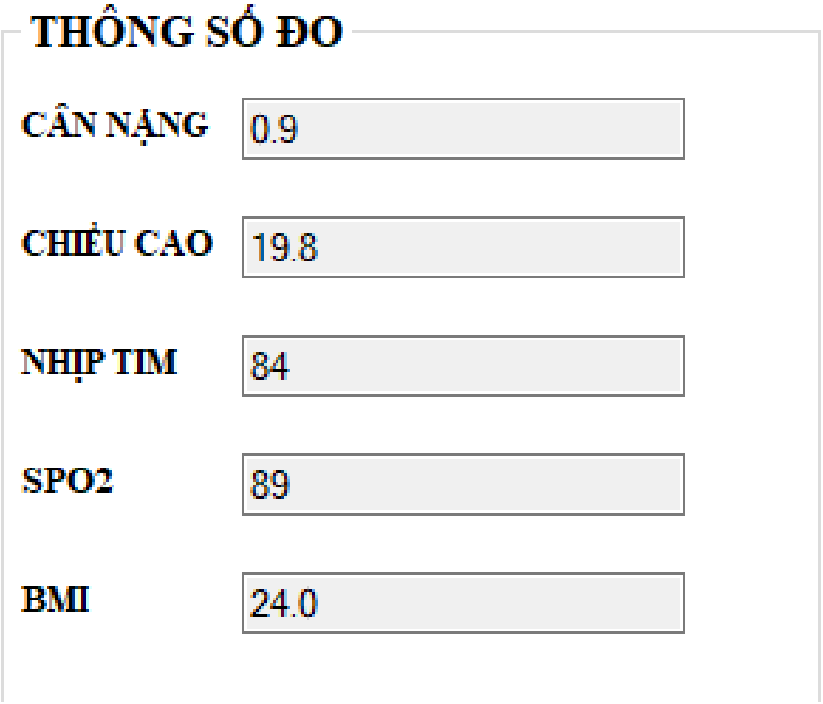


Figure 4.7. Result on PC



Figure 4.8. Result on LCD

4.2.4. Step 4: History Logging, PDF Export, and Printing

In the final stage, the nurse reviews the measured health indicators for accuracy. Upon clicking the "Xuất PDF" button, the software utilizes the iText7 library to generate a profssional medical report. The generated PDF file opens automatically on the desktop, allowing the nurse to click the print icon and provide a physical copy of the health report to the patient to complete the process.

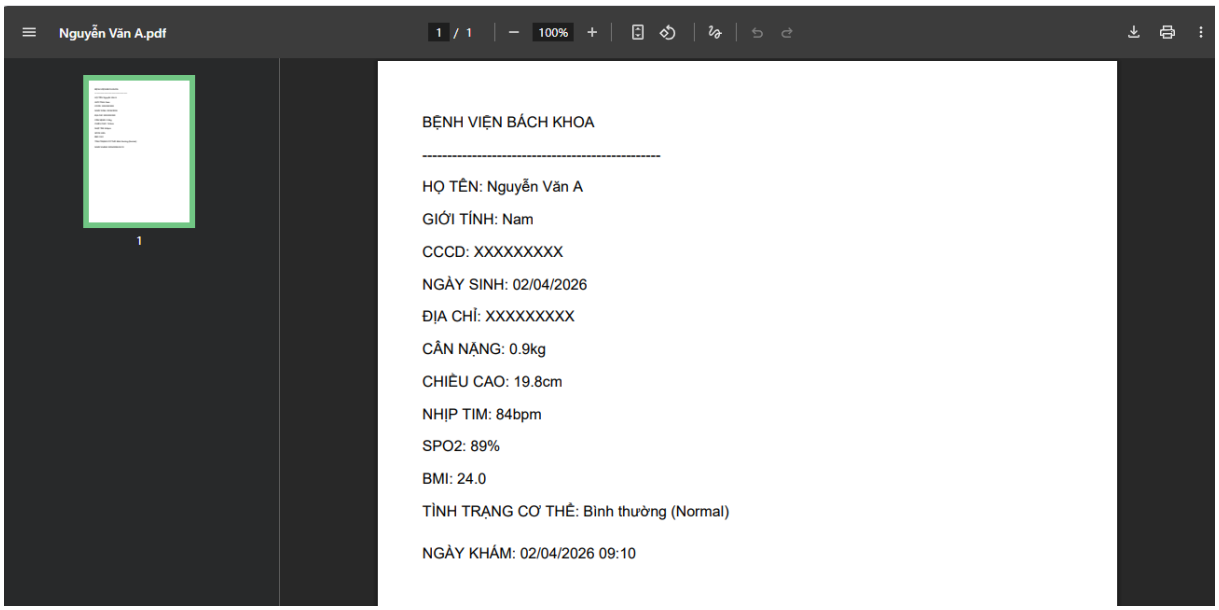


Figure 4.9. Print PDF

Simultaneously, the patient's record is saved into the history database for long-term tracking. By clicking the "Lịch sử" button on the main dashboard, a secondary window emerges, displaying a comprehensive list of all previous measurements.

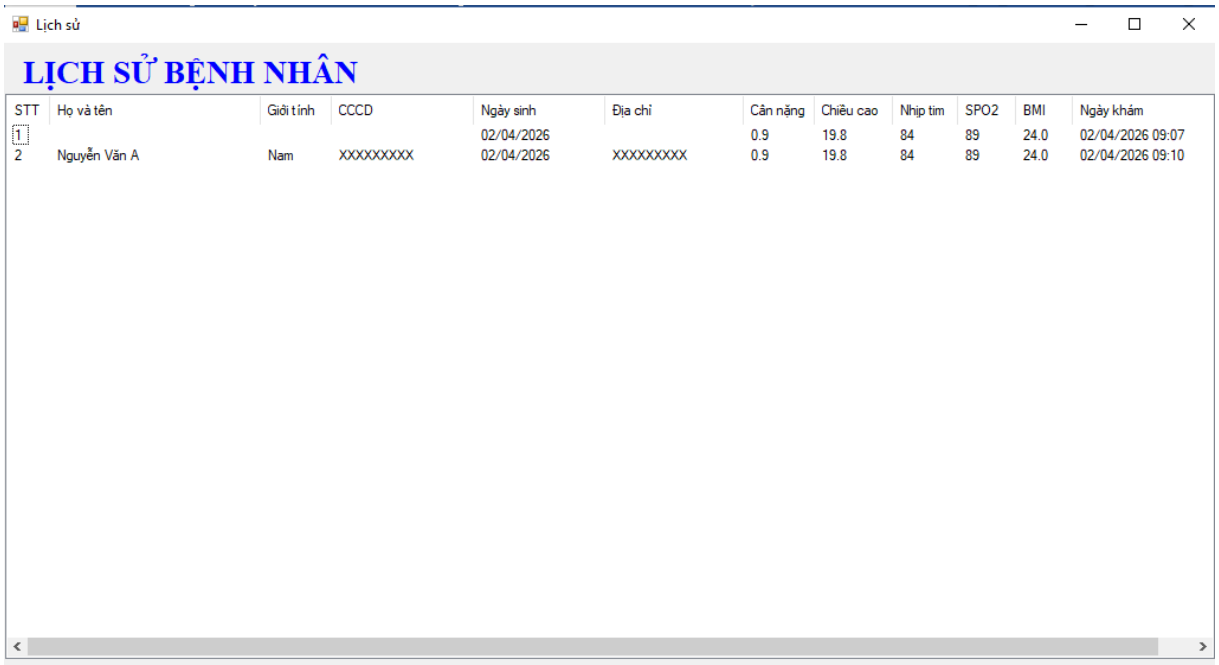


Figure 4.10. History logs

4.3. Conclusion

4.3.1. Results

- Hardware Design Process: Researching project components and analyzing their datasheets to thoroughly understand their technical specifications and operational principles.
- Component Selection: Selecting appropriate sensors for the project requirements; assigning microcontroller I/O pins to sensors and actuators based on their specific functional characteristics.
- PCB Design and Fabrication: Designing the circuit using Altium Designer software followed by the fabrication process. This stage requires extreme precision at every step to prevent errors that would necessitate restarting the entire fabrication cycle.
- Library Development: Since Altium's default libraries may not include all required parts, custom Schematic symbols, PCB footprints, and 3D models were created manually based on component datasheets.
- Layout Design: Focusing on critical parameters such as via clearance, trace spacing (clearance), and component placement to ensure optimal board organization and consistency with schematic logic.
- Embedded Software (CCS C): Mastered and effectively operated functional modules on the PIC16F877A microcontroller. The system maximizes hardware resources such as Timers and Timer Interrupts to ensure real-time performance. Communication protocols including I2C and UART are accurately configured for sensor data acquisition and PC communication.
- Management Software (C# WinForms): Successfully built an intuitive user interface using C#. The software performs key functions efficiently: scanning QR codes for patient identification, real-time UART communication, database management, and professional report generation in PDF format via the iText7 library

4.3.2. Techical Issues during Poduct Finalization

- Component Placement: The arrangement of components was not yet optimized relative to the dimensions of the copper board.
- PCB Routing: The routing process has not reached full optimization for a single-layer (Bottom layer) design, necessitating the use of jumper wires on the Top layer.
- Budget Constraints: Due to a limited student budget, high-precision components were not utilized. Currently, the project focuses on demonstrating and validating the core conceptual ideas at a prototype level.
- Sensor Limitations: The Loadcell used is a 1kg variant, intended only for simulation purposes and is not yet capable of measuring actual human weight.

- The HC-SR04 ultrasonic sensor is currently positioned to align with the existing physical model; therefore, it has not yet been verified for measuring actual human height.
- The system operates stably with high reliability and features intelligent patient detection; however, the program memory usage is relatively high due to extensive global variables and floating-point calculations.

4.3.3. Future Development

- Additional Health Monitoring: Integrating blood pressure measurement capabilities into the existing system.
- Data Redundancy: Implementing real-time storage registers to ensure data integrity and prevent loss in the event of a UART-RS232 connection failure.
- Mobile Integration & AI Analytics: Developing a dedicated mobile application for real-time health monitoring, integrated with AI-driven analysis to provide personalized lifestyle recommendations and structured wellness roadmaps.

REFERENCES

[1] Dang Phuoc Vinh, Vo Nhu Thanh, "PIC microcontroller engineering textbook", Construction Publishing House, 2019.

[2] PhD. Dang Phuoc Vinh, MSc. Tran Quang Khai, PhD. Doan Le Anh, PhD. Vo Nhu Thanh, PhD. Pham Anh Duc, "Programming PIC microcontrollers and peripherals", Science and Technology Publishing House, 2021.

[3] Microchip, “PIC16F87XA Datasheet”, 28/40/44-Pin Enhanced Flash Microcontrollers.

[4] Assoc. Prof. Dr Dang Phuoc Vinh, “Industrial Sensor”, 2025 – 2026.

[5] Elijah J. Morgan, “HC-SR04 Ultrasonic Sensor”, Nov. 16 2014

[6] “Ultrasonic Sensor HC-SR04 and Arduino – Complete Guide”, [Online], Available: https://howtomechatronics.com/tutorials/arduino/ultrasonic-sensor-hc-sr04/ [Accessed: April 3, 2026]

[7] Sparkfun, “Load Cell Amplifier HX711 Breakout Hookup Guide”, [Online], Available: https://www.alldatasheet.com/datasheet-pdf/view/1371088/SPARKFUN/HX711.html , [Accessed: April 3, 2026]

[8] Avia Semiconductor Co., Ltd. “HX711 Datasheet”, [Online], Available: https://www.alldatasheet.com/datasheet-pdf/pdf/1132222/AVIA/HX711.html , [Accessed: April 3, 2026]

[9] Maxim integrated, “MAX30102 Datasheet”, [Online], Available: Mixed-signal and digital signal processing ICs | Analog Devices , [Accessed: April 3, 2026]

[10] “Interfacing MAX30102 Pulse Oximeter and Heart Rate Sensor with Arduino”, [Online], Available: https://lastminuteengineers.com/max30102-pulse-oximeter-heart-rate-sensor-arduino-tutorial/?utm_source=chatgpt.com , [Accessed: April 3, 2026]

[11] “MAX30102 Sensor with Arduino”, [Online], Available: https://circuitdigest.com/microcontroller-projects/how-max30102-pulse-oximeter-and-heart-rate-sensor-works-and-how-to-interface-with-arduino?utm_source=chatgpt.com , [Accessed: April 3, 2026]

[12] Texas Instruments, “ Max232 Datasheet”, [Online], Available: MAX232 Dual EIA-232 Drivers and Receivers datasheet (Rev. N) , [Accessed: April 3, 2026]

[13] “Keypads 4x4 Matrix Membrane Keypad Datasheet”, [Online], Available: https://www.mouser.vn/ProductDetail/Parallax/27899?qs=qy7CnzY7nG6lIJGLMsv55A%3D%3D&srsltid=AfmBOooXsTrn45nhkBUYrCUDgcC5igQEwfNl2jJ6ncLHOZp6p78SsfqP , [Accessed: April 3, 2026]

[14] “Altium Designer Documentation”, [Online], Available: https://www.altium.com/documentation/altium-designer?srsltid=AfmBOopG-7-HNrwQdLPCF_JUmOfYCwtqBBOj4Zs7OZRtYkaAxWzCnaoZ, [Accessed: April 3, 2026]